

SA2 - Vulnerability Assessment Report

Case Information

Company Analyzed: BriteSpark

Analyst ID: 523883

Date of Investigation:

Organization: Mega Breach LTD.

Abstract— This report presents a static and dynamic analysis of SampleMal.exe, a Trojan Downloader with credential theft capabilities. The malware uses XOR-based obfuscation to hide embedded FTP credentials and payload information, decrypted only during execution. Using Ghidra and a controlled INetSim lab, the sample was observed connecting to ftp.adrive.com, retrieving secondary payloads, and executing them. Process Monitor and Wireshark revealed additional behaviors such as file system manipulation, registry access, and self-deletion. These findings confirm the malware’s dual purpose: downloading external payloads and exfiltrating data through stealthy and anti-forensics techniques, enabling persistent compromise. To mitigate such threats, the report recommends strict egress filtering, application whitelisting, and deploying EDR solutions with behavioral analytics. This analysis highlights the value of combining static and dynamic techniques in malware investigations and the importance of layered defenses to protect against advanced persistent threats.

Keywords: Trojan Downloader, XOR Decryption, Malware Analysis, Payload Execution, Credential Theft

TABLE OF CONTENTS

1. INTRODUCTION	3
2. METHODOLOGY	4
2.1 Task Environment and Setup	4
2.1.1 Virtual Machine Setup	4
2.1.2 Analysis Network Configuration	4
2.1.3 Precautionary Measures:.....	5
2.2 Tools Used	5
2.2.1 Host-Based Analysis Tools (FLARE VM)	6
2.2.2 Network-Based Analysis Tools (Remnux)	7
3. INVESTIGATION.....	8
3.1 Static Analysis:	8
3.1.1 File Hashing and External Sources Comparison (Cmder)	8
3.1.2 Strings Extraction (FLOSS).....	10
3.1.3 PE Structure Inspection (CFF Explorer).....	12
3.1.4 Import Address Table (IAT) Analysis	13
3.1.5 Packing Detection Analysis	17
3.1.6 Decryption Process and Credential Recovery Advanced Static (IDA & Ghidra)	19
3.2 Dynamic Analysis	24
3.2.1 Simulated Network Analysis (Wireshark & INetSim)	24
3.2.2 Host-Based Dynamic Analysis (Procmon)	26
3.2.2.1 Setup and Filtering	26
3.2.2.2 Key Host Behaviors Identified.....	26
3.2.3 Network Redirection and Traffic Interception (Procmon & Netcat)	29
4. SUMMARY OF FINDINGS, ANALYSIS, AND DISCUSSION.....	31
4.1 Summary of Discussion.....	31
4.2 Summary of Findings and Analysis	32
4.2.1 Main Function (_main)	32
4.2.2 Function sub_401000 (Decryption Function)	32
4.2.3 Function sub_401090 (Payload Downloader and Executor)	33
4.2.4 Function sub_401170 (Initialization and Preparation)	34
5. RECOMMENDATIONS FOR PREVENTIVE CONTROLS	35
5.1 Runtime Memory Scanning (Advanced EDR Tools)	35
5.2 FTP Traffic Monitoring and Blocking.....	35
5.3 Application Whitelisting.....	36
5.4 Network Segmentation and Egress Filtering.....	36
6. CONCLUSION.....	38
7. REFERENCES	39
8. APPENDICES	42

1. INTRODUCTION

This report was prepared following a security exercise at SecBam, where a suspicious executable was revealed. As a Jr. Analyst of SecBam, a trusted cybersecurity and forensics firm, we immediately followed strict protocols and isolated the file to protect confidentiality and system integrity.

The analysis aims to assess the file's threat level, understand its behavior, and evaluate its potential impact on SecBam's infrastructure. It focuses on determining the file's general purpose, internal structure, and any malicious techniques it may employ. Insights from this investigation will enhance SecBam's defensive strategies.

A secure virtual environment enabled detailed static and dynamic analysis of the sample. The objectives included identifying its main functionality, locating deobfuscation routines, and evaluating host and network activities.

The investigation meets the depth expected of a Junior Intrusion Analyst, thorough, but without full reverse engineering. A triage methodology was used, beginning with quick static analysis to flag indicators of compromise, followed by deeper evaluations (Atak, 2024). The analysis incorporated a variety of cybersecurity tools, which are detailed in the following sections.

2. METHODOLOGY

2.1 Task Environment and Setup

A carefully controlled environment was created to ensure both effective analysis and complete isolation from the external network to prevent contamination of the malicious file.

2.1.1 *Virtual Machine Setup*

Virtualization Software: Oracle VM VirtualBox Manager.

Guest Operating Systems:

- **Windows 10:** Installed with FLARE VM, which provides reverse engineering and malware analysis tools (Gomez, Stancill and Raabe, 2022).
- **REMnux:** Deployed as a second virtual machine focused on capturing and analyzing network-based indicators (MITRE, 2024).

2.1.2 *Analysis Network Configuration*

To isolate and safely analyze the file “SampleMal.exe”, a private host-only network was configured for safe practice:

Adapter Type: Host-Only Adapter.

IP Configuration:

- Host IP Address: 10.0.0.1
- Subnet Mask: 255.255.255.0
- Server Address: 10.0.0.2
- Server Mask: 255.255.255.0
- Lower Address Bound: 10.0.0.3
- Upper Address Bound: 10.0.0.254

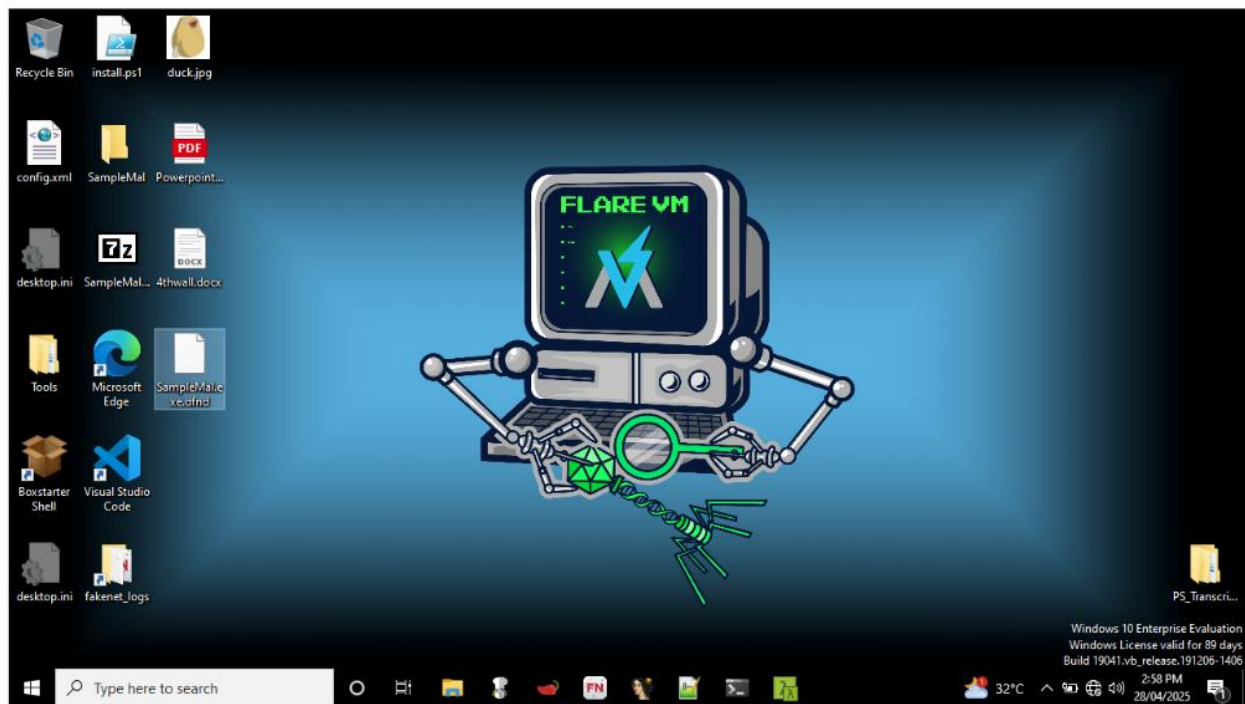
Adapter Settings:

Both FLARE VM and REMnux VM were configured with Host-Only Adapters only, disabling NAT to isolate the malware analysis environment fully and prevent external communication.

2.1.3 Precautionary Measures:

The suspicious file was defanged (e.g., renamed to SampleMal.exe.dfnd) to avoid accidental execution. Three decoy files (duck.jpg, 4thwall.docx, powerpoint.pdf) were added to monitor for file tampering or exfiltration behavior.

Figure 1: FlareVM base-install setup



2.2 Tools Used

A range of tools was employed, categorized into Host-Based and Network-Based tools, with specific uses throughout the analysis phase:

2.2.1 Host-Based Analysis Tools (FLARE VM)

Figure 2: Table of Host-Based Analysis tools

Tool	Usage	Rationale
Cmder	Terminal emulator is used to navigate the environment and run basic commands.	Provides a customizable and efficient command-line interface for initial file handling (U.S. Department of Veterans Affairs, 2019).
File Hashes	SHA256 and MD5 were computed for the sample.	Used to verify file integrity, check known malware databases, and compare versions (Rival and Yi, 2020).
VirusTotal	Submitted the hash or binary to detect known signatures.	Confirms whether the sample has been previously detected by antivirus vendors (CISA, 2022).
FLOSS	Extracted hardcoded strings, including obfuscated ones.	Helps reveal network addresses, credentials, or commands embedded in the binary (Jaime and Barata, 2022).
CFF Explorer	Viewed PE header details like entry point, imports, and sections.	Essential for understanding file structure, execution behavior, and identifying packers (Arfeen et al., 2021).
Ghidra	Disassembled the binary, located XOR key, and encrypted data.	Allows detailed static reverse engineering and code flow analysis.
IDA	Cross-referenced with Ghidra for key functions (e.g., FUN_00401000).	Used for validation and exploring complex control structures or function calls.

Joe Sandbox Cloud	Submitted the malware for automated static and dynamic behavior reporting.	Provides high-level behavioral summary and IOCs for rapid triage and reporting (Bencherchali, 2020).
Procmon (Static)	Used to simulate execution in a controlled environment and trace file/registry access statically.	Reveals potential behavior before runtime, such as keys or files the malware touches (Russovich, 2023).

2.2.2 Network-Based Analysis Tools (Remnux)

Figure 3: Table of Network-Based Analysis Tools

Tool	Usage	Rationale
Wireshark	Captured network traffic during execution.	Identified FTP and HTTPS connections, exfiltration attempts, and possible C2 traffic (Wireshark Foundation, 2024).
INetSim	Simulated common internet services like FTP and HTTP locally.	Ensures malware connects to a fake, controlled server instead of the real internet.
Netcat	Monitored incoming/outgoing connections manually.	Useful for confirming socket usage or receiving shell/command connections (NMAP, 2019).
Procmon (Dynamic)	Monitored real-time file, registry, and process events during execution.	Confirmed the creation/deletion of files, self-deletion, registry access, and persistence (O'Reilly Media, Inc., 2025).

3. INVESTIGATION

3.1 Static Analysis:

Various tools were used to examine the executable without running it, revealing its structure, behavior, and potential threats.

3.1.1 File Hashing and External Sources Comparison (Cmder)

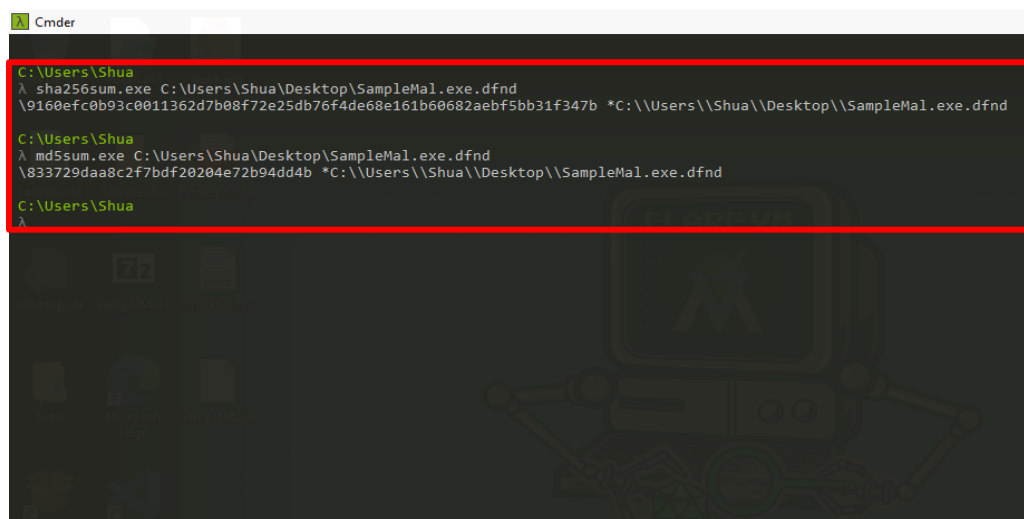
SHA-256 and MD5 hashes were generated via Cmder and checked against VirusTotal (2019) & Joe Sandbox to identify any known malware matches.

Figure 4: Table of Hashes

Hash Algorithm	Computed Hash
MD5	833729daa8c2f7bdf20204e72b94dd4b
SHA-256	9160efc0b93c0011362d7b08f72e25db76f4de68e161b60682aebf5bb31f347b

The file was hashed locally to ensure the sample was not altered and to prevent direct file submission to external servers, maintaining the confidentiality of SecBam's data policies.

Figure 5: Hash Output in Cmder

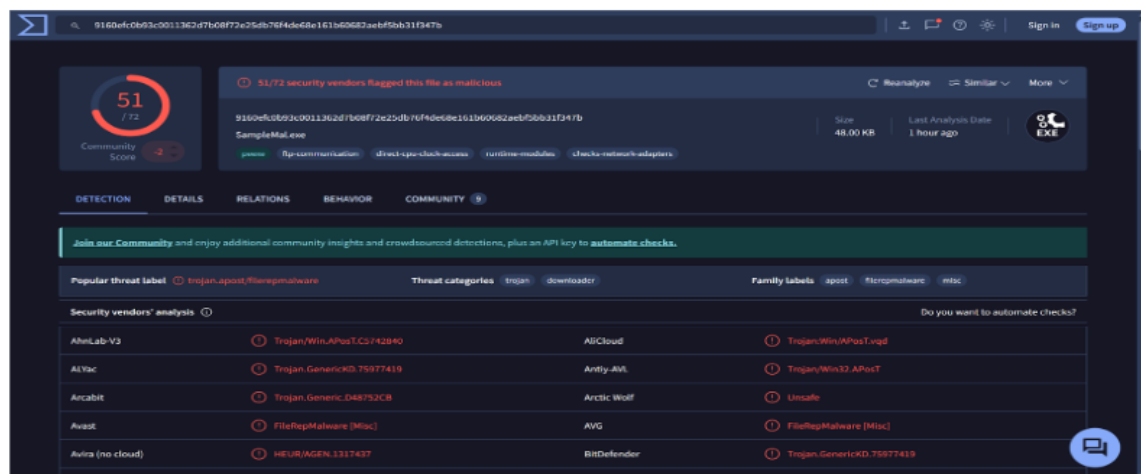


```
Cmder
C:\Users\Shua
λ sha256sum.exe C:\Users\Shua\Desktop\SampleMal.exe.dfnd
9160efc0b93c0011362d7b08f72e25db76f4de68e161b60682aebf5bb31f347b *C:\Users\Shua\Desktop\SampleMal.exe.dfnd
C:\Users\Shua
λ md5sum.exe C:\Users\Shua\Desktop\SampleMal.exe.dfnd
833729daa8c2f7bdf20204e72b94dd4b *C:\Users\Shua\Desktop\SampleMal.exe.dfnd
C:\Users\Shua
λ
```

Subsequently, the hashes were checked against VirusTotal:

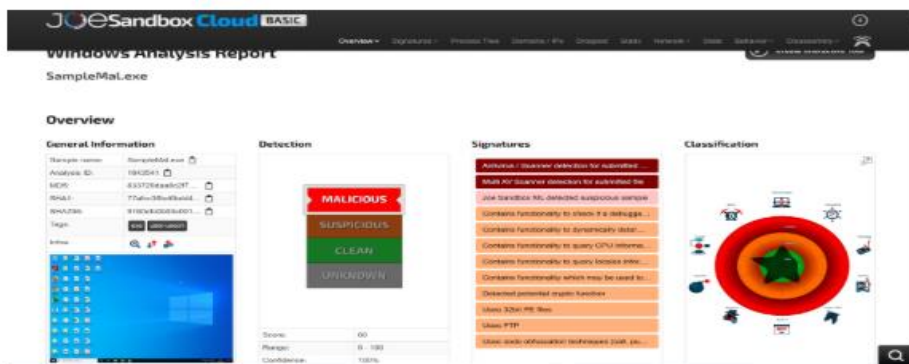
- The MD5 hash (833729daa8c2f7bdf20204e72b94dd4b) was recognized by VirusTotal and flagged by multiple antivirus engines as malicious.
- Similarly, the SHA-256 hash (9160efc0b93c0011362d7b08f72e25db76f4de68e161b60682aebf5bb31f347b) was found in VirusTotal's database and associated with known malware samples.

Figure 6: VirusTotal Website Overview



This external validation strengthens the initial suspicion that the file was indeed malicious, as matches were shown in the VirusTotal scan and Joe Sandbox Cloud website (engine detections, threat names, and detection rates) (Stoeski, 2025).

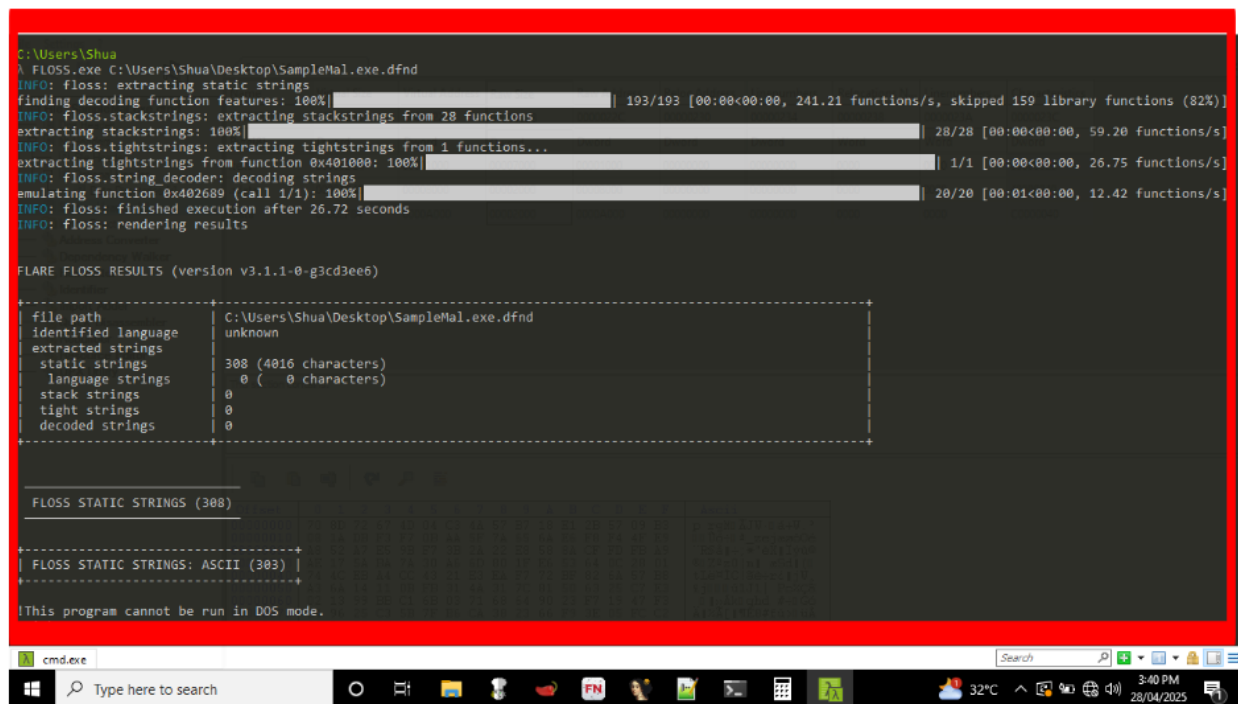
Figure 7: JoeSandboxCloud Website Overview



3.1.2 Strings Extraction (FLOSS)

FLOSS was used to extract hidden and plaintext strings from the binary, revealing obfuscated data, critical API calls, and potential malware behavior indicators. The results of the analysis are displayed below:

Figure 8: FLOSS Output in Cmdr



```
C:\Users\Shua
> FLOSS.exe c:\Users\Shua\Desktop\SampleMal.exe.dfn
INFO: floss: extracting static strings
finding decoding function features: 100%| 193/193 [00:00<00:00, 241.21 functions/s, skipped 159 library functions (82%)]
INFO: floss.stackstrings: extracting stackstrings from 28 functions
extracting stackstrings: 100%| 28/28 [00:00<00:00, 59.20 functions/s]
INFO: floss.tightstrings: extracting tightstrings from 1 functions...
extracting tightstrings from function 0x401000: 100%| 1/1 [00:00<00:00, 26.75 functions/s]
INFO: floss.string.decoder: decoding strings
emulating function 0x402689 (call 1/1): 100%| 20/20 [00:01<00:00, 12.42 functions/s]
INFO: floss: finished execution after 26.72 seconds
INFO: floss: rendering results

FLARE FLOSS RESULTS (version v3.1.1-0-g3cd3ee6)

-----
| file path          | C:\Users\Shua\Desktop\SampleMal.exe.dfn |
| identified language| unknown                                |
| extracted strings  |                                           |
| static strings     | 308 (4016 characters)                  |
|   language strings | 0 ( 0 characters)                     |
|   stack strings    | 0                                       |
|   tight strings    | 0                                       |
|   decoded strings  | 0                                       |
|-----|-----|

FLOSS STATIC STRINGS (308)

-----
| FLOSS STATIC STRINGS: ASCII (303) |
|-----|-----|

!This program cannot be run in DOS mode.
```

1.) PE Structure and Obfuscation:

The file shows a standard PE signature but contains suspicious patterns (e.g., VVVVVV, PPPPP, SSSSS), suggesting junk data to hinder static analysis.

2.) Rich Header:

Built with Microsoft toolchains (Rich signature), a neutral but notable detail when combined with other indicators.

3.) Anti-Debugging Techniques:

APIs like IsDebuggerPresent, SetUnhandledExceptionFilter, TerminateProcess, and GetTickCount point to potential anti-analysis behavior.

4.) Dynamic Memory Manipulation:

Calls to VirtualAlloc, HeapAlloc, and HeapCreate suggest runtime payload decryption, code injection, or shellcode execution.

5.) Process and File Manipulation:

CreateProcessW, DeleteFileW, and GetModuleFileNameA indicate capabilities for process creation, file deletion, and persistence.

6.) Network Activity Indicators:

Imports from WININET.dll (e.g., InternetOpenW, InternetConnectW, FtpGetFileW) show network communication capabilities, likely for payload download, data exfiltration, or C2 interaction.

7.) CRT Runtime Errors:

Strings like R6034 and R6025 suggest possible manipulation of the C Runtime Library (CRT), a tactic used to evade detection or disrupt analysis.

8.) Packed or Noisy Data:

Random strings (e.g., MZu3, YYuTVWh) indicate possible packing, encryption, or dummy data to evade static detection.

9.) Suspicious API Usage:

Use of mscoree.dll alongside native libraries hints at potential .NET code integration to complicate reverse engineering

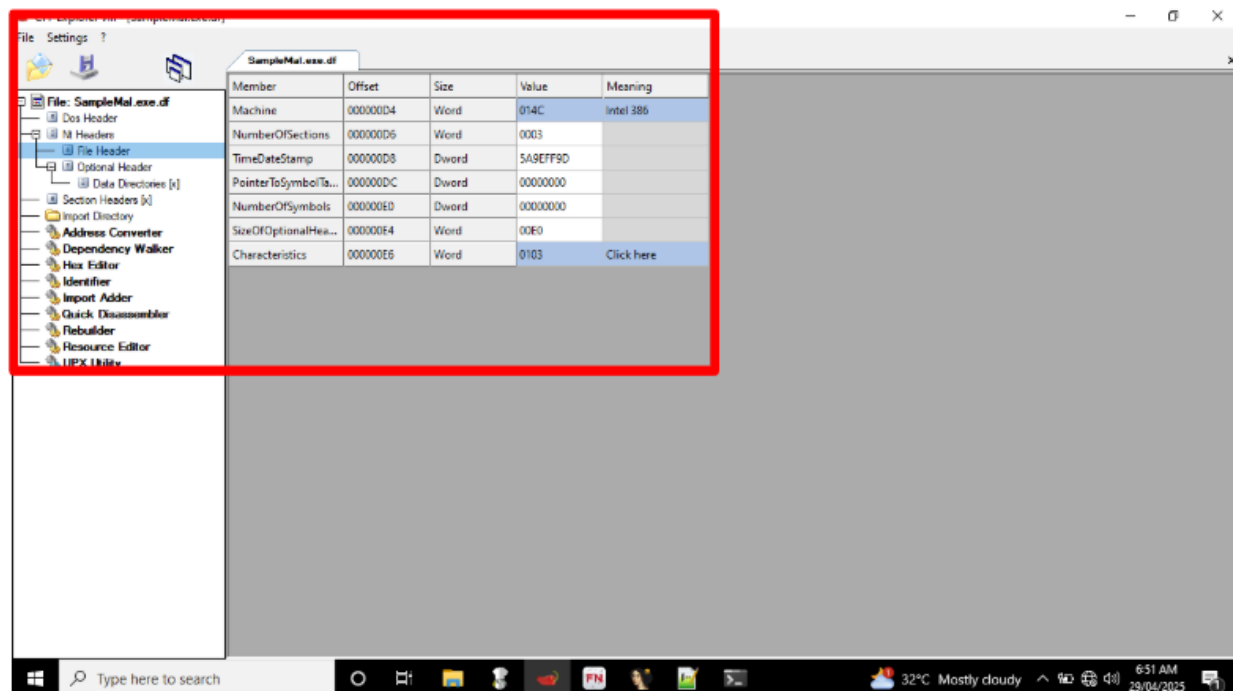
3.1.3 PE Structure Inspection (CFF Explorer)

Using CFF Explorer, a comprehensive inspection of the binary's Portable Executable (PE) structure was performed:

Figure 9: Table of PE Structure Inspection

Characteristic	Observation
DOS Header	Magic number "MZ" <u>identified</u> , confirming <u>the executable</u> runs in DOS mode initially, as standard Windows PE files do. Common in malware.
Time/Date Stamp	The TimeDateStamp field is set to 0x5A9EFF9D, which is 2018-03-08 20:47:25.
Section Analysis	The Virtual and Raw Size differences, especially the larger Virtual Size of ".rdata", suggest potential runtime unpacking or decryption, aligning with <u>evidence of memory manipulation and dynamic deobfuscation</u> .

Figure 10: CFF Explorer File Header Overview



3.1.4 Import Address Table (IAT) Analysis

The Import Directory analysis revealed extensive use of system-level APIs, particularly from KERNEL32.dll and WININET.dll, suggesting critical capabilities typically leveraged by malware:

Figure 11: CFF Explorer Import Directory Overview

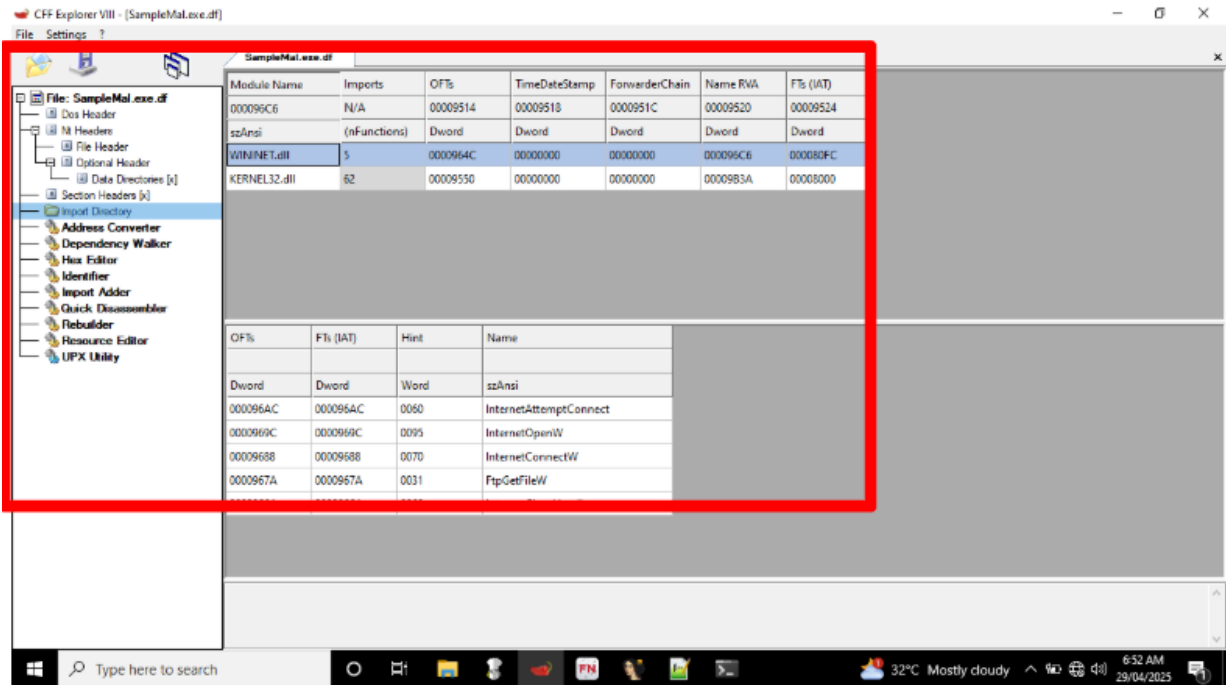


Figure 12: Table of API Functions and Malicious Usage

API Function	Potential Malicious Usage (per malAPI.io and MITRE ATT&CK)
InternetOpenW, InternetConnectW, InternetAttemptConnect,	T1071: Application Layer Protocol- Establish network connections for

UnhandledExceptionFilter, SetUnhandledExceptionFilter, RtlUnwind	T1562.001: Impair Defenses- Manipulate exception handling to disrupt analysis or hijack control flow.
IsDebuggerPresent	T1497.001: Virtualization/Sandbox Evasion- Detect debugger presence and alter behavior accordingly.
LoadLibraryA, GetProcAddress, GetModuleHandleA	T1055.001: Dynamic Linker Hijacking- Load and resolve additional modules and APIs dynamically to obfuscate malicious behavior.
WriteFile, GetStdHandle, GetFileType	T1005: Data from Local System- Potentially write exfiltrated or staged data to files or devices.
GetEnvironmentStringsA/W, FreeEnvironmentStringsA/W	T1083: File and Directory Discovery- Enumerate system environment variables for path manipulation or discovery.
InitializeCriticalSection, EnterCriticalSection, LeaveCriticalSection, DeleteCriticalSection, TlsAlloc, TlsGetValue, TlsSetValue, TlsFree, InterlockedIncrement, InterlockedDecrement	T1497: Virtualization/Sandbox Evasion- Thread-local storage and synchronization primitives can be used to obfuscate execution flow and complicate debugging.

InternetCloseHandle, FtpGetFileW (WININET.dll)	Command and Control (C2) or download additional payloads (Hassan, 2024).
CreateProcessW, DeleteFileW	T1105: Ingress Tool Transfer and T1561: Disk Wipe Launch new processes for payload execution and delete evidence (HP Development Company, L.P., 2025).
HeapAlloc, HeapFree, HeapCreate, HeapDestroy, HeapReAlloc, GetProcessHeap, HeapSize	T1027: Obfuscated Files or Information- Dynamic memory management for unpacking encrypted payloads at runtime.
VirtualAlloc, VirtualFree, VirtualProtect, VirtualQuery	T1055: Process Injection- Allocate and modify memory regions for code injection and execution.
GetTickCount, Sleep	T1497: Virtualization/Sandbox Evasion- Introduce execution delays to evade automated sandbox analysis (Miguel, 2025).
GetCurrentProcessId, GetCurrentProcess, TerminateProcess	T1489: Service Stop- Interact with and potentially terminate security-related processes.

GetLocaleInfoA, LCMapStringA/W, MultiByteToWideChar, WideCharToMultiByte, GetCPInfo, GetACP, GetOEMCP, GetStringTypeA/W	T1071: Application Layer Protocol (with possible localization targeting)- Adapt execution based on system locale, which can tailor attacks by region.
--	---

Interpretation:

Even in the absence of full dynamic execution, the imported functions suggest the sample exhibits:

- **Dynamic memory manipulation** (heap and virtual memory control),
- **Network communication capabilities** (potential C2 or payload retrieval),
- **Anti-debugging and anti-analysis behavior** (sandbox evasion techniques),
- **Process manipulation and file system interaction** (process injection, file deletion),
- **Dynamic API resolution and thread management** (obfuscation and resilience mechanisms).

The breadth and nature of imported APIs strongly suggest the sample has moderate to high risk if executed, indicating functionality common in malware focused on persistence, defense evasion, payload delivery, and system manipulation.

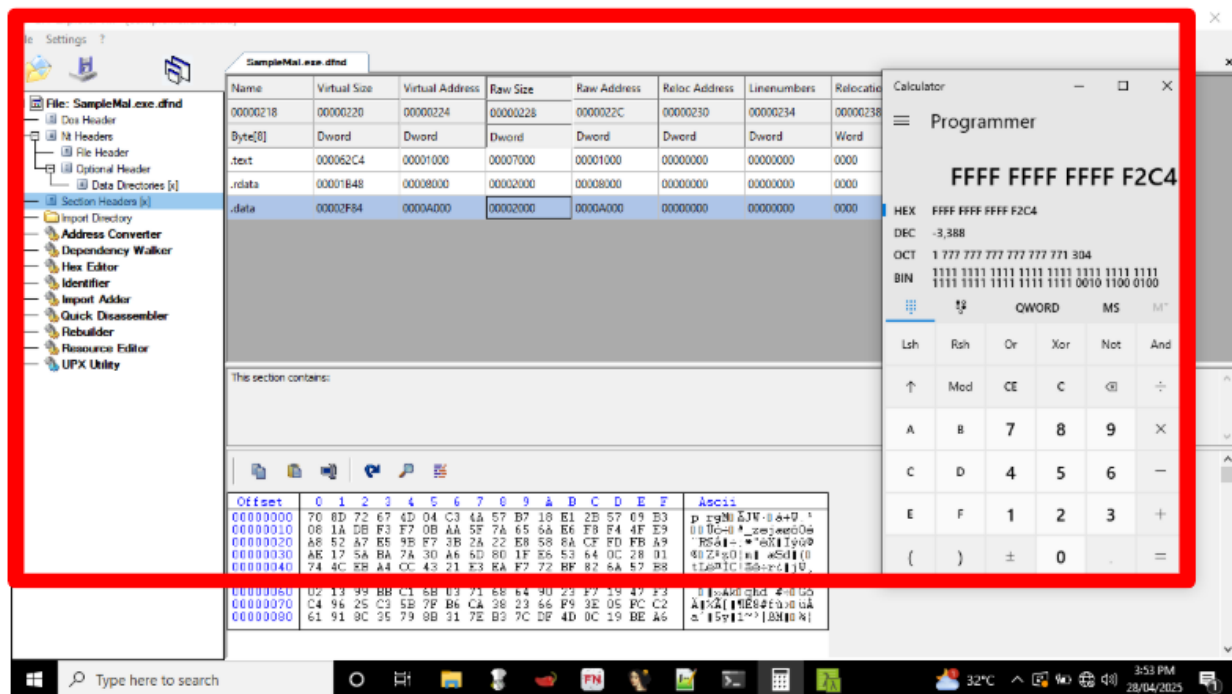
Figure 13: MITRE ATT&CK Matrix of SampleMal.Exe

Mitre Att&ck Matrix

Reconnaissance	Resource Development	Initial Access	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	Discovery	Lateral Movement	Collection	Command and Control	Exfiltration	Impact
Gather Victim Identity Information	Acquire Infrastructure	Valid Accounts	1 Native API	1 DLL Side-Loading	1 Process Injection	1 Process Injection	OS Credential Dumping	1 System Time Discovery	Remote Services	1 Archive Collected Data	1 Encrypted Channel	1 Exfiltration Over Alternative Protocol	Abuse Accessibility Features
Credentials	Domains	Default Accounts	Scheduled Task/Job	1 Boot or Logon Initialization Scripts	1 DLL Side-Loading	1 DLL Side-Loading	LSASS Memory	2 1 Security Software Discovery	Remote Desktop Protocol	Data from Removable Media	1 Non-Application Layer Protocol	Exfiltration Over Bluetooth	Network Denial of Service
Email Addresses	DNS Server	Domain Accounts	At	Logon Script (Windows)	Logon Script (Windows)	1 Obfuscated Files or Information	Security Account Manager	2 3 System Information Discovery	SMB/Windows Admin Shares	Data from Network Shared Drive	1 1 Application Layer Protocol	Automated Exfiltration	Data Encrypted for Impact

3.1.5 Packing Detection Analysis

Figure 14: CC Explorer Section Headers Overview & Programmer Calculator



The Virtual Size and Raw Size of the executable sections were inspected using CFF Explorer and calculated using Calculator (Programmer) to check for signs of executable packing or compression:

Figure 15: Table of Sections in SampleMal.exe & Sizes

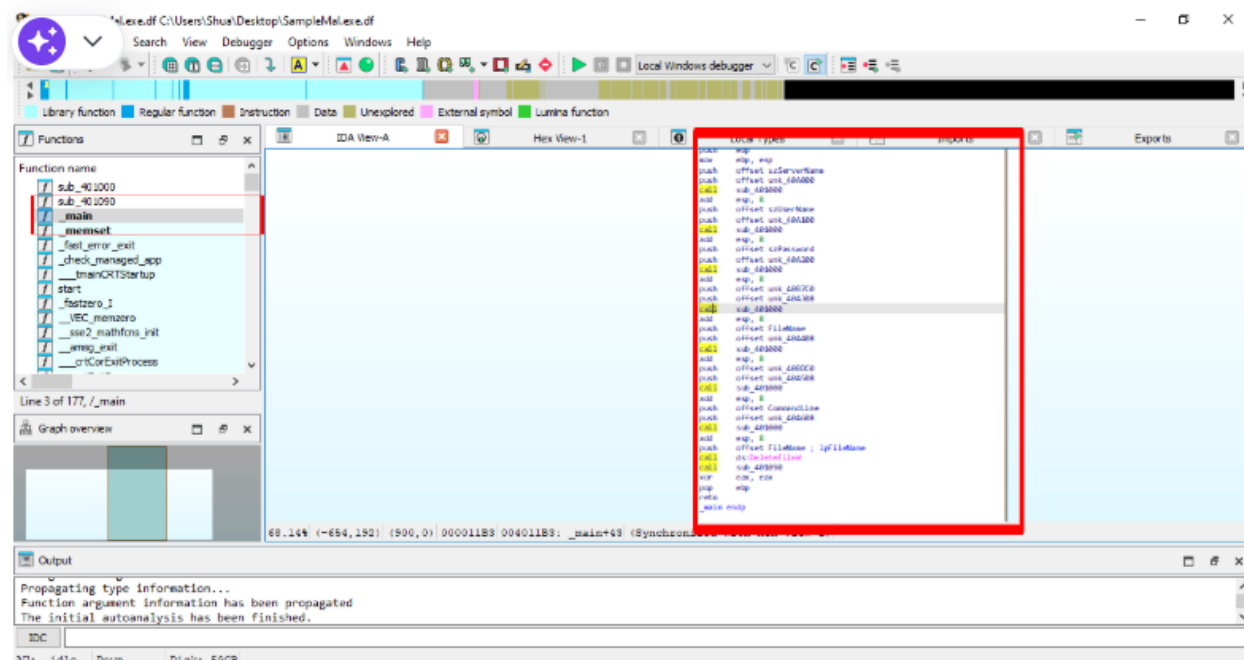
Section	Virtual Size (Hex)	Raw Size (Hex)	Virtual Size (Decimal)	Raw Size (Decimal)	Notes
.text	0x62C4	0x7000	25,508	28,672	Executable code section. Raw size > virtual size suggests padding (common). Padding can be used to hide injected shellcode or junk data.
.rdata	0x1B48	0x2000	6,472	8,192	Normal for a read-only data section. Raw size larger than the virtual size indicates standard alignment and padding. No major anomaly.
.data	0x2F84	0x2000	12,612	8,192	Suspicious: Virtual size > Raw size. Indicates the program allocates more writable memory at runtime than exists in the file, commonly used by malware for unpacking or runtime modification.

Interpretation:

The analysis reveals that the .text section has a larger Raw Size, suggesting the presence of padding without significant obfuscation. The .rdata section appears normal. The .data section shows a large difference, indicating padded data, which may reflect potential obfuscation.

3.1.6 Decryption Process and Credential Recovery | Advanced Static (IDA & Ghidra)

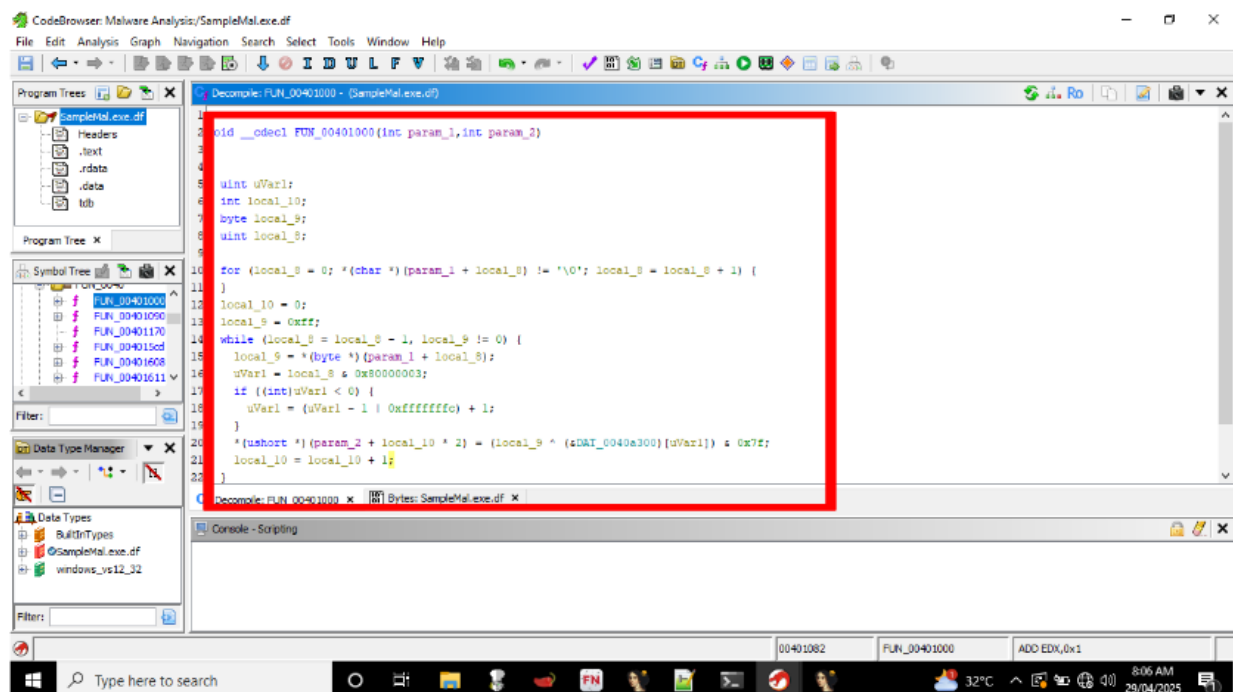
Figure 16: IDA Overview of `_main`



Using Interactive Disassembler (IDA), we identified that the `_main` function serves as the central coordinator of the malware's execution flow. It initializes the malware environment and proceeds to decrypt a set of embedded, obfuscated strings that are crucial for further stages of the attack. It does this by repeatedly invoking a specific subroutine located at `0x401000` (commonly labeled `sub_401000`), which handles the decryption of sensitive configuration data.

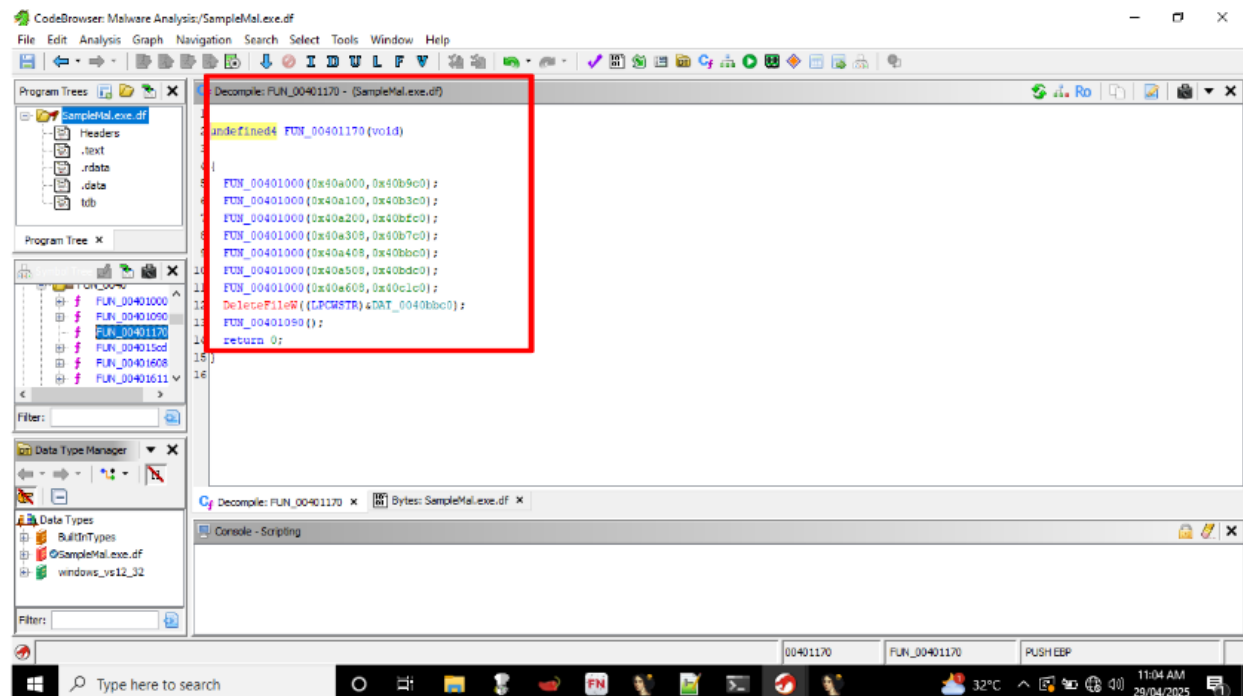
These encrypted elements include fields such as `szServerName`, `szUserName`, `szPassword`, `FileName`, and `CommandLine`. Once the required information is decrypted in memory, `_main` proceeds to execute a series of follow-up actions. Notably, it calls `DeleteFileW` to erase temporary or staging files, likely used to conceal its tracks. Following this cleanup step, the function invokes another subroutine, `sub_401090`, which is responsible for the actual execution of the decrypted command or payload.

Figure 17: Ghidra Decompiler of FUN_00401000



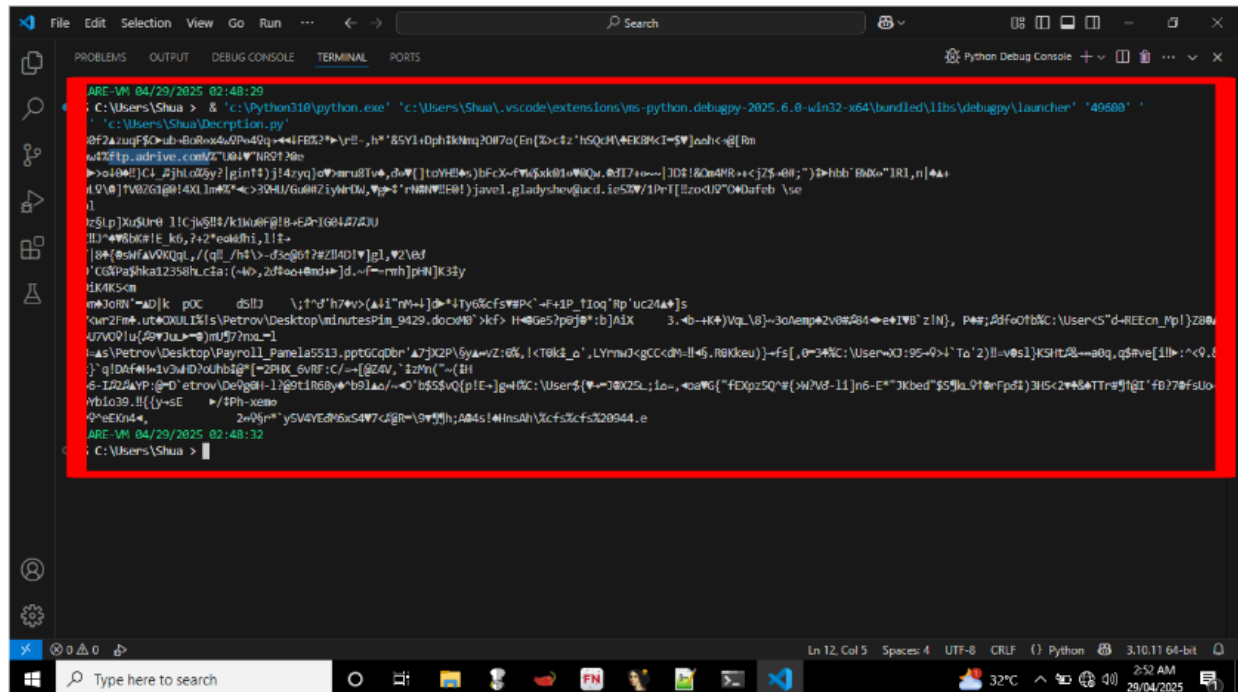
The investigation began with reverse-engineering the function `FUN_00401000`, which was suspected of handling decryption routines within the malware. Static analysis via Ghidra confirmed that this function performed XOR operations using a key stored at memory address `0x40A300`. The XOR key was extracted directly from the disassembled code after observing it being used to manipulate string data in memory. The bytes at this location were assumed to be critical for decoding encrypted content embedded within the executable.

Figure 18: Ghidra Decompiler of FUN_00401170



To locate the encrypted data blocks, further analysis was conducted on the function `FUN_00401170`, which was invoked during early malware execution. This function referenced memory locations such as `0x40A000`, `0x40A100`, and `0x40A200`, all of which appeared to contain obfuscated strings or binary segments. Since this function was responsible for initializing core malware behavior, these data blocks were determined to be central to its payload execution.

Figure 19: Virtual Studio Overview of Decrypted Content



To recover the hidden data, a Python script was developed in Visual Studio Code. The decryption logic was built based on an XOR algorithm, with the structure and approach refined through guidance from ChatGPT (OpenAI, 2025). The script parsed the relevant memory regions and applied the XOR key byte-by-byte to reconstruct the original plaintext strings. This approach successfully decrypted a series of credentials and payload indicators.

Figure 20: Table of Decrypted Content

Field	Decrypted Content
FTP Host	ftp.adrive.com
Username(s)	Petrov, pavel.gladyshec@ucd.ie

Password	Pa\$hka123
Searched File Names from \Petrov\Desktop*	minutesPim_9429.docx
	Payroll_Pamela5513.ppt
	receipt_Diana_377.pptx

Interpretation:

This set of strings confirmed that the malware was configured to exfiltrate specific files or download secondary payloads from the FTP server using these credentials.

An example is a particularly revealing decrypted string was \Petrov\Desktop\Payroll_Pamela5513.ppt. This indicated a specific local file path under the user profile “Petrov”, which aligns with the FTP username and suggests that the malware either targets this user or operates within this user context. The file name “Payroll_Pamela5513.ppt” suggests an intent to search for or extract financial documents, particularly those involving employee payroll or sensitive presentation files.

It becomes clear that this malware was designed to extract specific files of interest and transmit them externally. The presence of real usernames, passwords, and file names indicates deliberate, targeted behavior rather than generic malware spread.

3.2 Dynamic Analysis

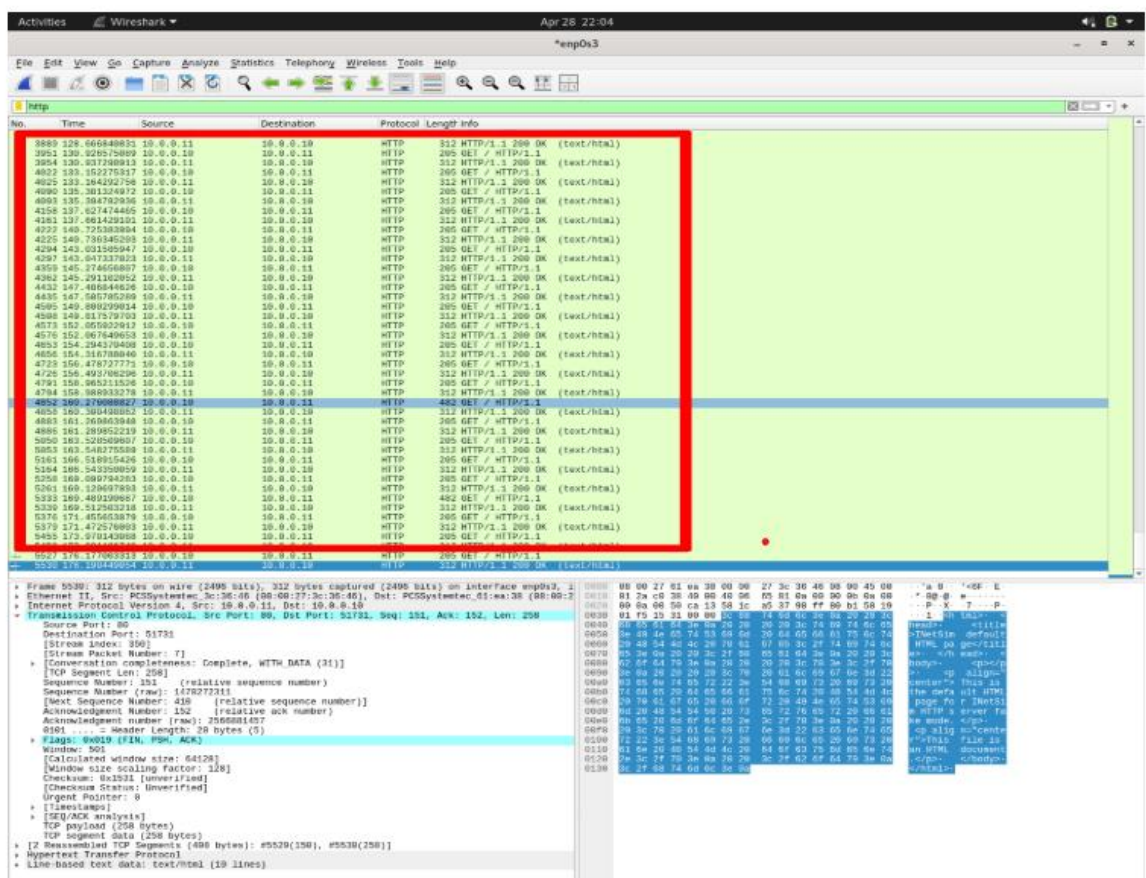
3.2.1 Simulated Network Analysis (Wireshark & INetSim)

- **Network Monitoring Setup:**

INetSim simulated services (HTTP, HTTPS, FTP, DNS) locally, and Wireshark captured network traffic to safely observe malware behavior without internet exposure.

- **Packet Capture and Analysis:**

Figure 21: Wireshark Overview of Filter: HTTP



(1) Wireshark logs showed the malware attempted an HTTPS connection to an external domain, which we assume is ftp.adrive.com, outside INetSim's simulation.

Figure 22: Wireshark Overview of Filter: FTP

[illegible]

(2) Additionally, network analysis revealed an FTP connection used to exfiltrate sensitive credentials, including the password Pa\$hka123.

Interpretation:

Encrypted outbound traffic indicated potential C2 communication or data exfiltration. Although the connection failed thanks to containment, the behavior matched typical post-compromise tactics.

3.2.2 Host-Based Dynamic Analysis (Procmon)

Procmon (Process Monitor) from Sysinternals was utilized to capture host-based indicators and monitor the behavior of the executable in real-time during dynamic analysis.

3.2.2.1 Setup and Filtering

To efficiently isolate the malware's behavior and avoid overwhelming noise from system processes, targeted **filters** were applied:

- **Filter 1:**
 - **Condition:** Process Name → **is** → “SampleMal.exe”
 - **Purpose:** Focus exclusively on operations performed by the malicious process.
- **Filter 2:**
 - **Condition:** Operation → **contains** → “File”
 - **Purpose:** Display only file-related operations, such as file creation, modification, or deletion.

Additionally, the Process Tree feature was activated to observe parent-child relationships, helping to detect if the malware spawned additional processes or injected into legitimate ones, such as conhost.exe spawning from SampleMal.exe.

3.2.2.2 Key Host Behaviors Identified

Legend:

- **SUSPICIOUS:** 
- **POSSIBLY SUSPICIOUS:** 

Figure 23: Analysis & Interpretation Table of Host Behaviors

Behavior	Registry/File/System Component	Suspicious Indicator	Analysis and Interpretation
Registry Query	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Internet Settings\TcpAutotuning		Indicates potential attempts to inspect or manipulate TCP autotuning settings. This may be used to alter network behavior for stealthy communications or to optimize command-and-control (C2) traffic.
Registry Query	DNS-related keys (e.g., DNS timeout parameters)		Suggests the malware may be modifying DNS resolution behavior to evade detection, accelerate callback attempts, or support Fast Flux techniques.
File Creation	Files written or modified within C:\Windows\SysWOW64\ (e.g., DLLs such as cryptbase.dll)		Writing to system directories, especially SysWOW64, is highly suspicious. This behavior may indicate DLL side-loading, hijacking, or persistent code execution.
File Mapping	CreateFileMapping and MapViewOfFile involving DLLs		Memory-mapped files involving DLLs can be used for stealthy in-memory execution, code injection, or fileless malware operations.
Registry Access	Repeated use of RegOpenKey and RegCloseKey on TCP/IP parameters or autorun keys		Indicates reconnaissance activity. May precede registry modification to establish persistence, disable security features, or modify startup behavior.

File Access	Creation or modification of files in temporary or unexpected locations		May indicate payload staging or unpacking of components. Requires correlation with process behavior and file content.
--------------------	--	--	---

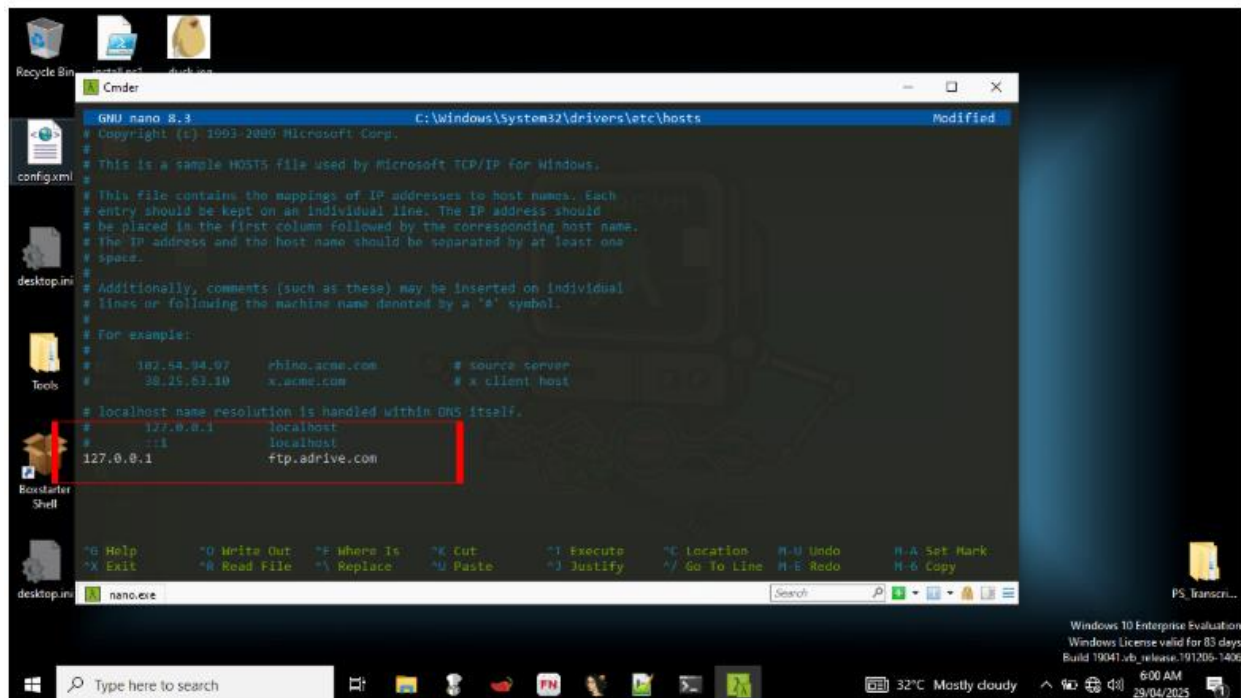
Interpretation:

The behaviors observed in SampleMal.exe, including modification of system directories, registry queries targeting network and DNS settings, and use of memory mapping, are consistent with known malware tactics. These may facilitate stealthy communication, system manipulation, persistence, and code injection.

3.2.3 Network Redirection and Traffic Interception (Procmon & Netcat)

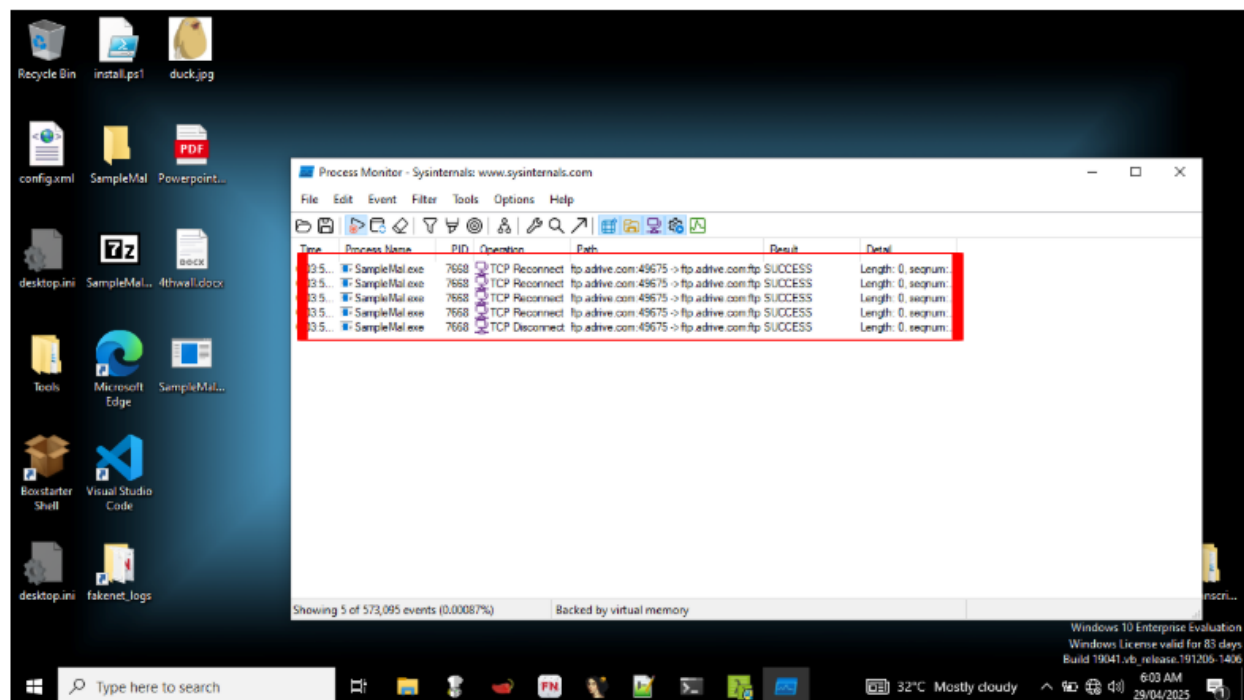
To analyze potential command-and-control (C2) communication and trick the malware into revealing network-based behavior, I modified the local hosts file using nano in Cmdr by appending the line 127.0.0.1 ftp.adrive.com at C:\Windows\System32\drivers\etc\hosts.

Figure 24: Cmdr Overview of Nano



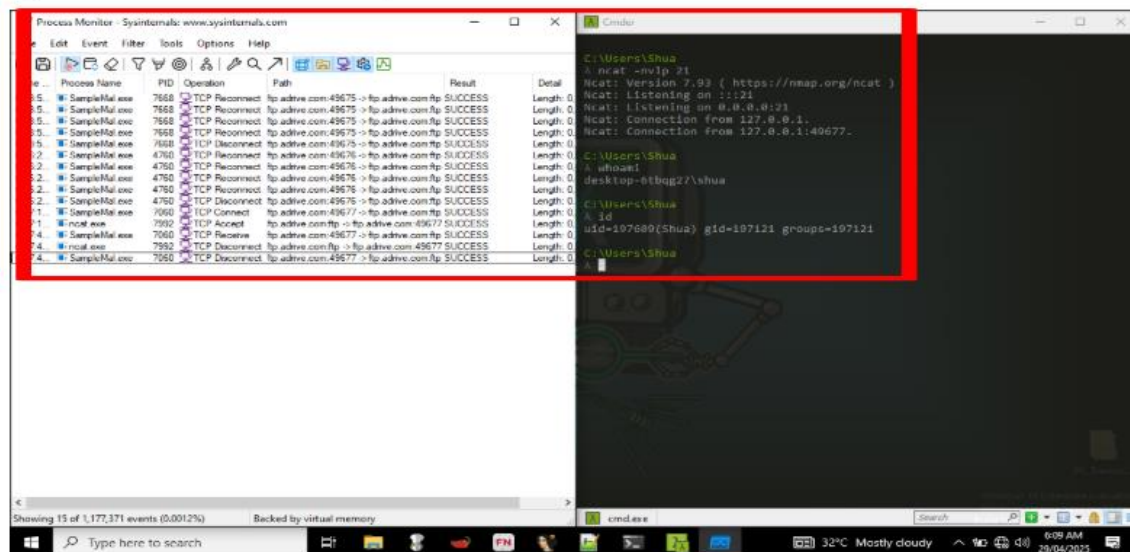
This redirected all outbound requests to ftp.adrive.com back to the local machine, effectively simulating the presence of the expected remote server. To monitor this behavior, Procmon was configured with a filter where Operation contains TCP, allowing observation of network-level interactions initiated by the malware. This confirmed that the binary attempts a DNS resolution for ftp.adrive.com, suggesting a hardcoded A Record callout.

Figure 25: ProcMon Overview of Initial Detonation



Additionally, a listener was established on port 21 using `ncat -nvlp 21` to emulate an FTP server and capture any attempted connections. This setup provided evidence of potential outbound network activity and may indicate the malware's behavior under controlled redirection, including attempts to retrieve payloads or exfiltrate data via FTP.

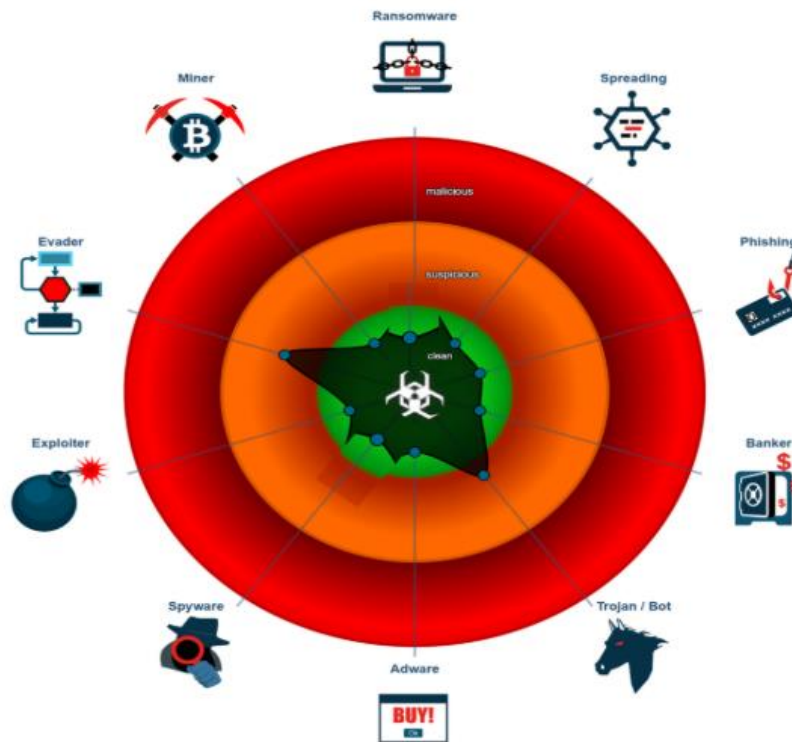
Figure 26: ProcMon & Cmdr Overview of NetCat



4. SUMMARY OF FINDINGS, ANALYSIS, AND DISCUSSION

4.1 Summary of Discussion

Figure 27: Classification Graph of SampleMal.exe



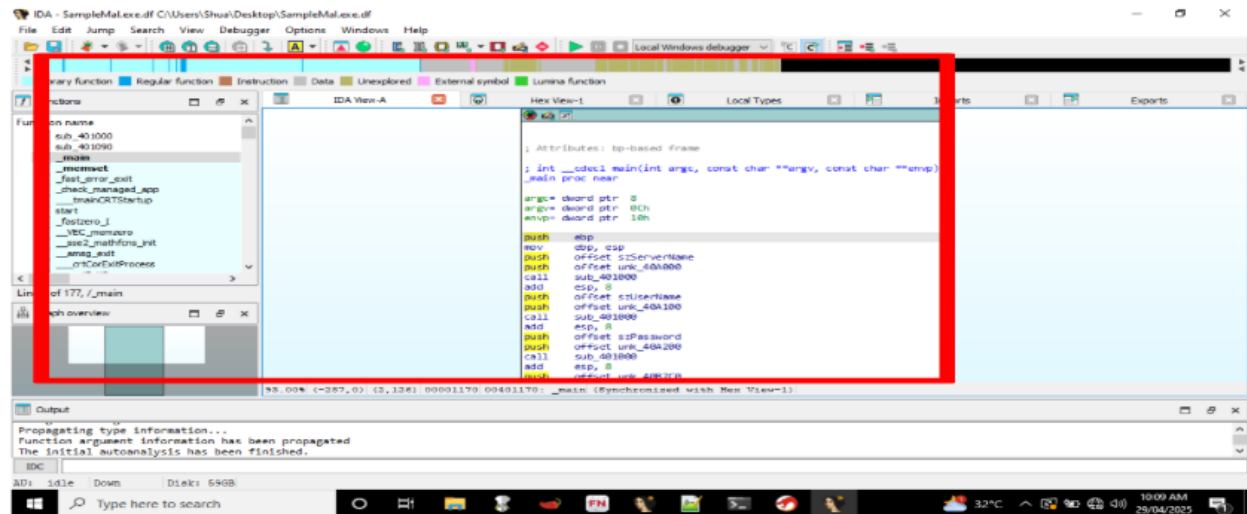
The analyzed malware is a Trojan Downloader with credential-stealing capabilities specifically targeting Petrov and their credentials. Its purpose is to decrypt hardcoded strings at runtime to retrieve a payload via FTP. It uses a custom XOR function (*sub_401000*) to obfuscate server addresses, credentials, and commands (FireEye, 2025). The *main()* function handles decryption, self-deletion (*DeleteFileW*), and triggers *sub_401090* to connect to the FTP server, download, and execute the payload. Anti-forensics measures include file deletion and system environment checks.

Dynamic analysis confirmed attempts to contact *ftp.adrive.com*, download files, and establish persistence. Network monitoring detected suspicious HTTPS traffic, while ProcMon revealed malware-related file creation and deletion, confirming the sample's behavior in a controlled environment.

4.2 Summary of Findings and Analysis

4.2.1 Main Function (_main)

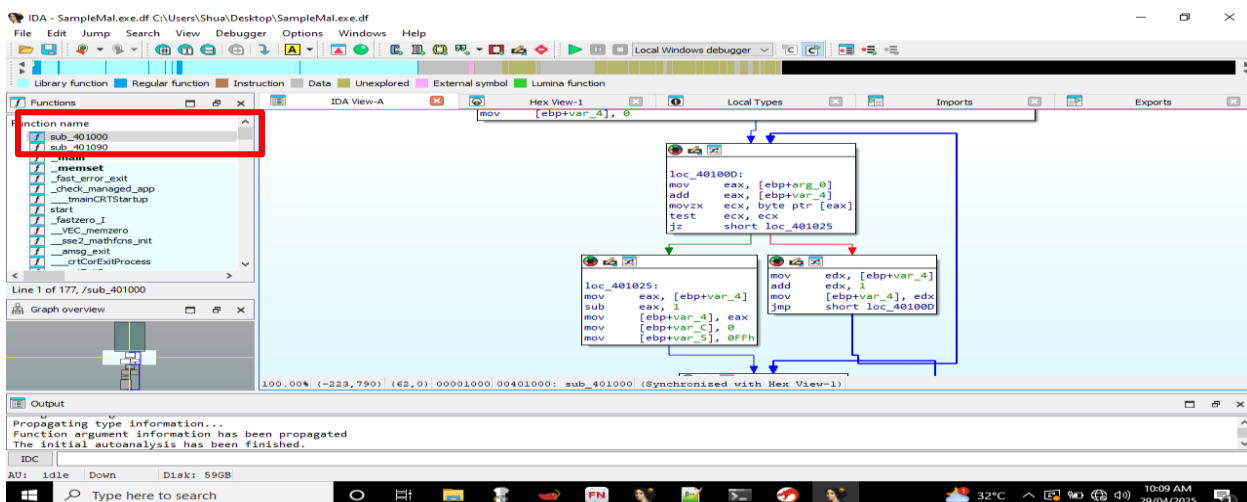
Figure 28: IDA Overview of _main



The `_main` function decrypts sensitive data using `sub_401000`, deletes the original file with `DeleteFileW`, and executes `sub_401090` for further malicious operations, initiating the malware's core behaviors: decryption, cleanup, and execution.

4.2.2 Function sub_401000 (Decryption Function)

Figure 29: IDA Overview of Function sub_401000



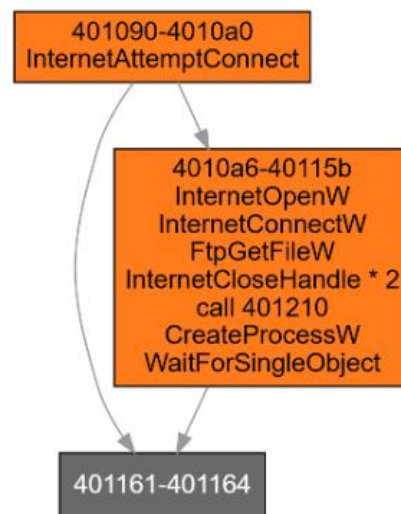
sub_401000 decrypts strings via reverse traversal and XOR operations on a static array (*0x40a300*). Static analysis showed that stack-based operations and masking techniques were used to hide credentials and commands until runtime, evading detection.

4.2.3 Function *sub_401090* (Payload Downloader and Executor)

Legend:

- EXECUTED: ■
- NOT EXECUTED: ■■

Figure 30: Control-Flow Graph of Function *sub_401090*



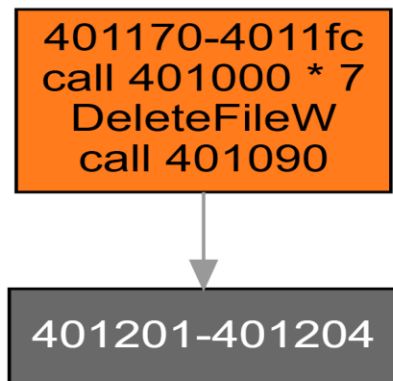
After cleanup, the malware uses *sub_401090* to establish an FTP connection, download a payload with decrypted credentials, and execute it via *CreateProcessW*, using *WaitForSingleObject* to monitor completion.

4.2.4 Function *sub_401170* (Initialization and Preparation)

Legend:

- EXECUTED: ■
- NOT EXECUTED: ■

Figure 31: Control-Flow Graph of Function *sub_401170*



sub_401170 repeatedly calls *sub_401000* to decrypt hidden data, deletes files, and triggers *sub_401090*. Initiated by `_tmainCRTStartup`, it ensures system compatibility before payload download and execution, maintaining stealth.

5. RECOMMENDATIONS FOR PREVENTIVE CONTROLS

While SecBam already has a strong technical control framework, additional specific improvements targeting threats of this nature are recommended:

5.1 Runtime Memory Scanning (Advanced EDR Tools)

Description:

Deploy Endpoint Detection and Response (EDR) tools capable of runtime memory scanning to capture decrypted values and detect malicious behaviors in memory before execution (Trend Micro, 2025).

Example Implementation:

- Tools like CrowdStrike Falcon or SentinelOne can inspect running processes, identify abnormal decryptions in memory, and quarantine files instantly (Trend Micro, 2025).
- Real-time memory scanning would detect deobfuscated FTP credentials and malicious payloads before they can act.

5.2 FTP Traffic Monitoring and Blocking

Description:

Strictly monitor or block FTP traffic at the network perimeter. Modern malware often uses FTP/SFTP for payload retrieval (Malwarebytes, 2020).

Example Implementation:

- Configure firewalls (e.g., Cisco ASA, Palo Alto) to block outgoing FTP unless explicitly allowed.
- Monitor FTP session attempts in SIEM platforms like Splunk or Elastic Security.
- Set anomaly detection alerts for unexpected FTP activity.

5.3 Application Whitelisting

Description:

Application whitelisting is a proactive security approach that permits only pre-approved, trusted software to run on a system (Shacklett, 2025). Instead of attempting to detect malicious applications, this method blocks all software by default unless explicitly authorized. Tools like Microsoft AppLocker or Windows Defender Application Control (WDAC) can be used to define rules based on file path, publisher, or file hash (Yuceel, 2022).

Example Implementation:

In a Windows environment, an organization can configure AppLocker policies via Group Policy to allow only digitally signed software from Microsoft and specific vendors. For example:

```
New-AppLockerPolicy -Path "C:\ApprovedApps\" -RuleType Publisher -User Everyone -XML
```

This ensures that any unsigned or unknown executables, such as those dropped by malware, will not be executed, significantly reducing the attack surface.

5.4 Network Segmentation and Egress Filtering

Description:

Network segmentation involves dividing a network into smaller, isolated segments to prevent lateral movement of malware (Wickramasinghe, 2024). Egress filtering restricts outbound traffic from endpoints or servers to only necessary destinations. When combined, these controls limit the ability of malware to communicate with command-and-control (C2) servers or exfiltrate data (Megabeets, 2025).

Example Implementation:

- **Segmentation:** Use VLANs or firewalls to separate workstations, servers, and critical systems. For example, FTP services should only be accessible from designated zones or hosts.

- **Egress Filtering:** Use a perimeter firewall or host-based firewall rules to allow outbound connections only to approved IPs/domains.
- **DNS Sinkholing:** Redirect known malicious domains (e.g., ftp.adrive.com in analysis) to localhost (127.0.0.1) by modifying the hosts file or using DNS firewall services like **Cisco Umbrella** or **Pi-hole**.

This technique was used during the malware analysis to trick the malware into communicating with a benign local endpoint, preventing actual external access.

6. CONCLUSION

This analysis confirms the malware is a Trojan Downloader with credential-stealing capabilities, designed to retrieve secondary payloads via FTP. It uses runtime decryption through the *sub_401000* function to conceal strings like server addresses and credentials. The *main()* function coordinates self-deletion and triggers FTP communication using *sub_401090*, showing anti-forensics and stealth features. Additionally, *sub_401170* plays a critical role in initialization and preparation, loading memory regions that contain encrypted configuration data essential to the attack.

Static tools like Ghidra and FLOSS revealed the malware's structure, while dynamic analysis with ProcMon and Wireshark confirmed file creation, deletion, and attempts to connect to *ftp.adrive.com*. The recovery of hardcoded credentials such as *Pa\$hka123* highlights its goal of data exfiltration and external command execution (Rusinovich, 2023).

This company-mandated exercise demonstrates the value of combining static and dynamic techniques to uncover complex malware behavior. It deepened understanding of threat tactics like string obfuscation, payload staging, and evasion methods. Identifying the username *Petrov* and the path *\Petrov\Desktop\Payroll_Pamela5513.ppt* suggests a targeted attack, possibly on a specific user or machine. To mitigate this, implementing User Behavior Analytics (UBA) and auditing the *Petrov* account is recommended (IBM, 2024).

To enhance SecBam's cybersecurity resilience, the following preventive measures are recommended:

- Deploy advanced Endpoint Detection and Response (EDR) tools with runtime memory scanning (Microsoft, 2024).
- Employ network segmentation to isolate critical systems from user workstations.
- Maintain a centralized logging and alerting system for real-time threat visibility.
- And of course, conduct continuous user training on phishing and social engineering threats.

These layered defenses will greatly improve SecBam's ability to detect, respond to, and neutralize similar threats in the future, preserving its reputation for security excellence.

7. REFERENCES

Arfeen, A., Ahmed, S., Khan, M.A. and Jafri, S.F.A. (2021). Endpoint Detection & Response: A Malware Identification Solution. *2021 International Conference on Cyber Warfare and Security (ICCWS)*, pp.1–8. doi:<https://doi.org/10.1109/iccws53234.2021.9703010>.

Atak, S.U. (2024). *Static Malware Analysis vs Dynamic Malware Analysis - Comparison Chart*. [online] Malwation.com. Available at: <https://www.malwation.com/blog/static-malware-analysis-vs-dynamic-malware-analysis-comparison-chart>.

Bencherchali, N. (2020). *Hunting Malware with Windows Sysinternals — Process Monitor*. [online] Medium. Available at: <https://nasbench.medium.com/hunting-malware-with-windows-sysinternals-process-monitor-e67476f44514>.

CISA (2022). *2021 Top Malware Strains | CISA*. [online] Cybersecurity and Infrastructure Security Agency CISA. Available at: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa22-216a>.

FireEye (2025). *About FireEye*. [online] fireeye.dev. Available at: <https://fireeye.dev/docs/about/fireeye/>.

Gomez, A.M.M., Stancill, B. and Raabe, M. (2022). *FLARE VM: A FLAREytale Open to the Public | Mandiant*. [online] Google Cloud Blog. Available at: <https://cloud.google.com/blog/topics/threat-intelligence/flarevm-open-to-public/>.

Hassan, N. (2024). *Application Layer: OSI Model's User Interface | Medium*. [online] Medium. Available at: <https://medium.com/@nay1228/mastering-the-application-layer-from-protocols-to-enterprise-implementation-6586972ecf39>.

HP Development Company, L.P. (2025). *Disk Wipe*. [online] Hp.com. Available at: https://support.hp.com/us-en/document/ish_6727361-3979072-16.

IBM (2024). *User behavior analytics*. [online] Ibm.com. Available at: <https://www.ibm.com/think/topics/user-behavior-analytics>.

Jaime, L. and Barata, J. (2022). *Using FLOSS to Implement COBIT for I&T Governance*. [online] ISACA. Available at: <https://www.isaca.org/resources/isaca-journal/issues/2022/volume-6/using-floss-to-implement-cobit-for-it-governance>.

Malwarebytes (2020). *2020 State of Malware Report PRESENTS*. [online] Available at: https://www.malwarebytes.com/wp-content/uploads/sites/2/2023/09/2020_state-of-malware-report.pdf.

Megabeets (2025). *Cutter*. [online] Cutter. Available at: <https://cutter.re/>.

Microsoft (2024). *Microsoft Learn: Build skills that open doors in your career*. [online] learn.microsoft.com. Available at: <https://learn.microsoft.com/en-us/>.

Miguel, P.G. (2025). *12 Best Virtualization Software in 2024*. [online] The CTO Club. Available at: <https://thectoclub.com/tools/best-virtualization-software/>.

MITRE (2024). *MITRE ATT&CK*. [online] Mitre.org. Available at: <https://attack.mitre.org/>.

Mossé Cyber Security Institute (2022). *Don't be Fooled by Malware in Disguise - Identifying Obfuscated Malware*. [online] Mosse-institute.com. Available at: https://library.mosse-institute.com/articles/2022/05/don-t-be-fooled-by-malware-in-disguise-identifying-obfuscated-malware/don-t-be-fooled-by-malware-in-disguise-identifying-obfuscated-malware.html?utm_source=chatgpt.com.

NMAP (2019). *Ncat*. [online] Nmap.org. Available at: <https://nmap.org/ncat/>.

O'Reilly Media, Inc. (2025). *Monitoring with Process Monitor - Practical Malware Analysis [Book]*. [online] www.oreilly.com. Available at: <https://www.oreilly.com/library/view/practical-malware-analysis/9781593272906/ch04s03.html>.

OpenAI (2025). *ChatGPT*. [online] chat.openai.com. Available at: <https://chat.openai.com/>.

Rival, X. and Yi, K. (2020). *Introduction to Static Analysis*. [online] Google Books. Available at: <https://books.google.ae/books?hl=en&lr=&id=-aLLDwAAQBAJ&oi=fnd&pg=PR7&dq=advance+static+analysis&ots=xmN8vFj97N&sig=KP>

pEVOpmL9zHnMhUeZ8nV9cwrLs&redir_esc=y#v=onepage&q=advance%20static%20analysis&f=false.

Russinovich, M. (2023). *Process Monitor - Windows Sysinternals*. [online] learn.microsoft.com. Available at: <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>.

Shacklett, M.E. (2025). *What is Application Whitelisting?* [online] SearchSecurity. Available at: <https://www.techtarget.com/searchsecurity/definition/application-whitelisting>.

Stoeski, D. (2025). *Joe Sandbox Authentication Guide | Tines Explained*. [online] Tines.com. Available at: <https://explained.tines.com/en/articles/8458932-joe-sandbox-authentication-guide>.

Trend Micro (2025). *Global AI Cybersecurity Leader*. [online] Trend Micro. Available at: https://www.trendmicro.com/en_ae/business.html?gad_source=1&gbraid=0AAAAAoI92weCeAu0AFcAQP4151o3uqtOB&gclid=Cj0KCQjwzrzABhD8ARIsANlSWNNBUib5cvC8eV8adeh_SgJfm2bzSNcv1u6UifTi4voEHCU_KdkO8FAaAoA9EALw_wcB.

U.S. Department of Veterans Affairs (2019). *Cmder*. [online] Va.gov. Available at: <https://www.oit.va.gov/Services/TRM/ToolPage.aspx?tid=14648>.

VirusTotal (2019). *VirusTotal*. [online] Virustotal.com. Available at: <https://www.virustotal.com/gui/home/upload>.

Wickramasinghe, S. (2024). *What is Network Segmentation? A Complete Guide | Splunk*. [online] Splunk. Available at: https://www.splunk.com/en_us/blog/learn/network-segmentation.html.

Wireshark Foundation (2024). *Wireshark*. [online] Wireshark.org. Available at: <https://www.wireshark.org/>.

Yuceel, H.C. (2022). *MITRE ATT&CK T1055 Process Injection*. [online] Picassecurity.com. Available at: https://www.picassecurity.com/resource/blog/t1055-process-injection?utm_source=chatgpt.com.

8 APPENDICES

Figure 1: FlareVM base-install setup

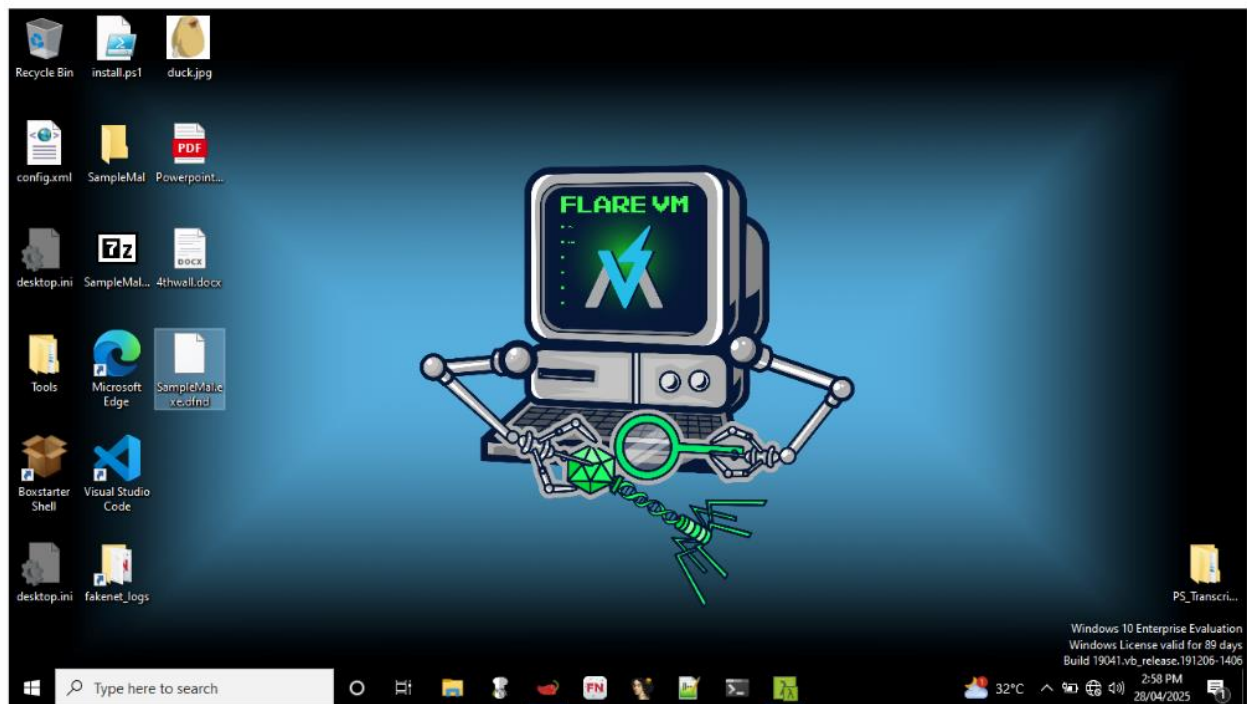


Figure 2: Table of Host-Based Analysis tools

Tool	Usage	Rationale
Cmder	Terminal emulator is used to navigate the environment and run basic commands.	Provides a customizable and efficient command-line interface for initial file handling (U.S. Department of Veterans Affairs, 2019).
File Hashes	SHA256 and MD5 were computed for the sample.	Used to verify file integrity, check known malware databases, and compare versions (Rival and Yi, 2020).
VirusTotal	Submitted the hash or binary to detect known signatures.	Confirms whether the sample has been previously detected by antivirus vendors (CISA, 2022).
FLOSS	Extracted hardcoded strings, including obfuscated ones.	Helps reveal network addresses, credentials, or commands embedded in the binary (Jaime and Barata, 2022).
CFF Explorer	Viewed PE header details like entry point, imports, and sections.	Essential for understanding file structure, execution behavior, and identifying packers (Arfeen et al., 2021).
Ghidra	Disassembled the binary, located XOR key, and encrypted data.	Allows detailed static reverse engineering and code flow analysis.
IDA	Cross-referenced with Ghidra for key functions (e.g., FUN_00401000).	Used for validation and exploring complex control structures or function calls.

Joe Sandbox Cloud	Submitted the malware for automated static and dynamic behavior reporting.	Provides high-level behavioral summary and IOCs for rapid triage and reporting (Bencherchali, 2020).
Procmon (Static)	Used to simulate execution in a controlled environment and trace file/registry access statically.	Reveals potential behavior before runtime, such as keys or files the malware touches (Rusinovich, 2023).

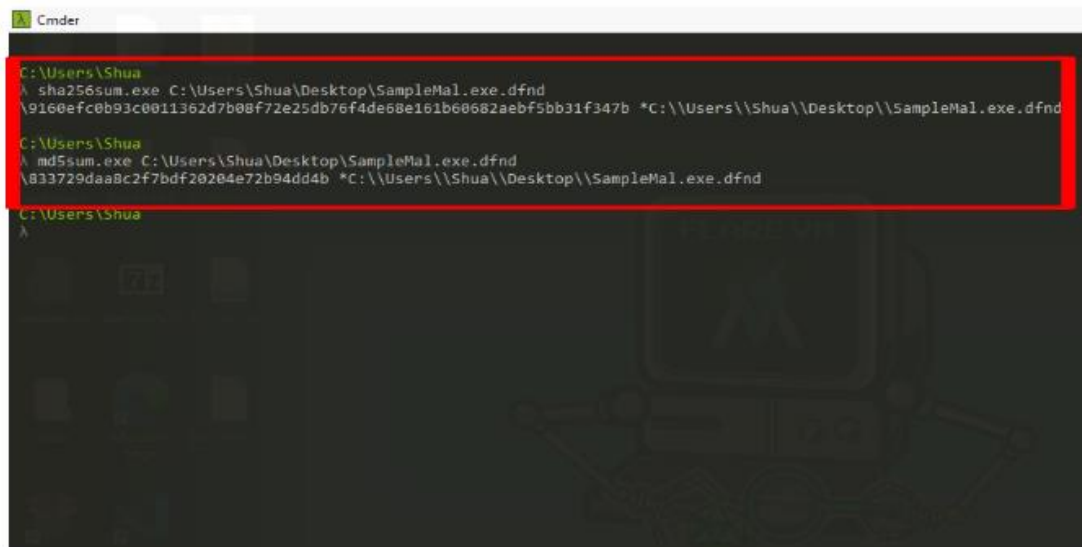
Figure 3: Table of Network-Based Analysis Tools

Tool	Usage	Rationale
Wireshark	Captured network traffic during execution.	Identified FTP and HTTPS connections, exfiltration attempts, and possible C2 traffic (Wireshark Foundation, 2024).
INetSim	Simulated common internet services like FTP and HTTP locally.	Ensures malware connects to a fake, controlled server instead of the real internet.
Netcat	Monitored incoming/outgoing connections manually.	Useful for confirming socket usage or receiving shell/command connections (NMAP, 2019).
Procmon (Dynamic)	Monitored real-time file, registry, and process events during execution.	Confirmed the creation/deletion of files, self-deletion, registry access, and persistence (O'Reilly Media, Inc., 2025).

Figure 4: Table of Hashes

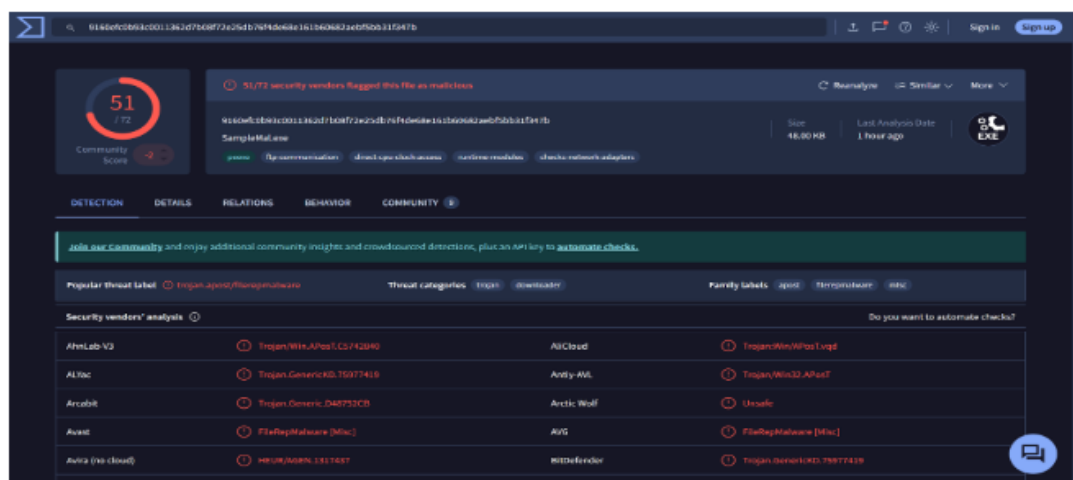
Hash Algorithm	Computed Hash
MD5	833729daa8c2f7bdf20204e72b94dd4b
SHA-256	9160efc0b93c0011362d7b08f72e25db76f4de68e161b60682aebf5bb31f347b

Figure 5: Hash Output in Cmder



```
Cmder
C:\Users\Shua
A sha256sum.exe C:\Users\Shua\Desktop\SampleMal.exe.dfn
9160efc0b93c0011362d7b08f72e25db76f4de68e161b60682aebf5bb31f347b *C:\Users\Shua\Desktop\SampleMal.exe.dfn
C:\Users\Shua
A md5sum.exe C:\Users\Shua\Desktop\SampleMal.exe.dfn
833729daa8c2f7bdf20204e72b94dd4b *C:\Users\Shua\Desktop\SampleMal.exe.dfn
C:\Users\Shua
A
```

Figure 6: VirusTotal Website Overview



The screenshot shows the VirusTotal website interface for a file named **SampleMal.exe**. The file's SHA256 hash is **9160efc0b93c0011362d7b08f72e25db76f4de68e161b60682aebf5bb31f347b**. The file size is **48.00 KB** and it was last analyzed **1 hour ago**. The community score is **51 / 72**. The file is identified as a **Trojan** with a **confidence** of **100%**. The file is categorized as **trojan**, **download**, and **trojan**. The file is also categorized as **trojan**, **download**, and **trojan**. The file is also categorized as **trojan**, **download**, and **trojan**.

Security vendors' analysis	Threat categories	Threat categories	Threat categories
AhnsLab-V3	Trojan:Win32/PSW.123456	AliCloud	Trojan:Win32/PSW.123456
ALYac	Trojan:Generic.KD.TG77419	Avast	Trojan:Win32/PSW.123456
Avast	Trojan:Generic.KD.TG77419	Avira	Trojan:Win32/PSW.123456
Avira (no cloud)	Trojan:Generic.KD.TG77419	BitDefender	Trojan:Win32/PSW.123456

Figure 7: JoeSandboxCloud Website Overview



Figure 8: FLOSS Output in Cmdr

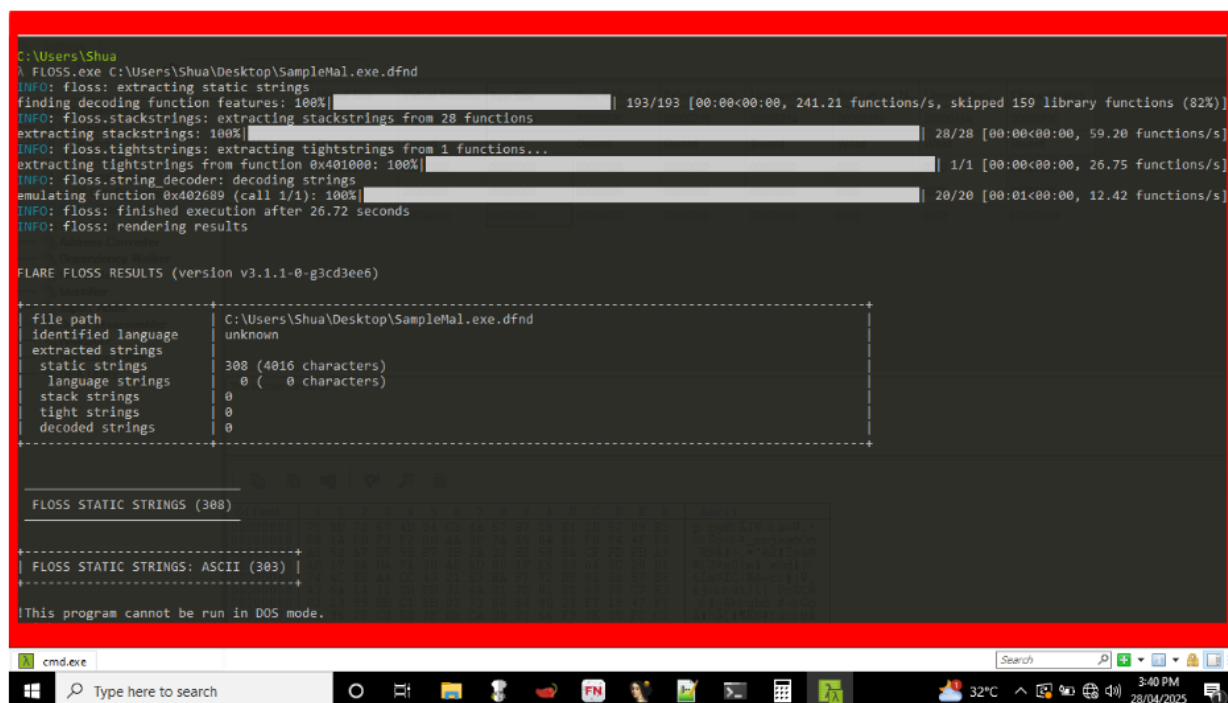


Figure 9: Table of PE Structure Inspection

Characteristic	Observation
DOS Header	Magic number "MZ" <u>identified</u> , confirming <u>the executable</u> runs in DOS mode initially, as standard Windows PE files do. Common in malware.
Time/Date Stamp	The TimeDateStamp field is set to 0x5A9EFF9D, which is 2018-03-08 20:47:25.
Section Analysis	The Virtual and Raw Size differences, especially the larger Virtual Size of ".rdata", suggest potential runtime unpacking or decryption, aligning with <u>evidence of</u> memory manipulation and dynamic deobfuscation.

Figure 10: CFF Explorer File Header Overview

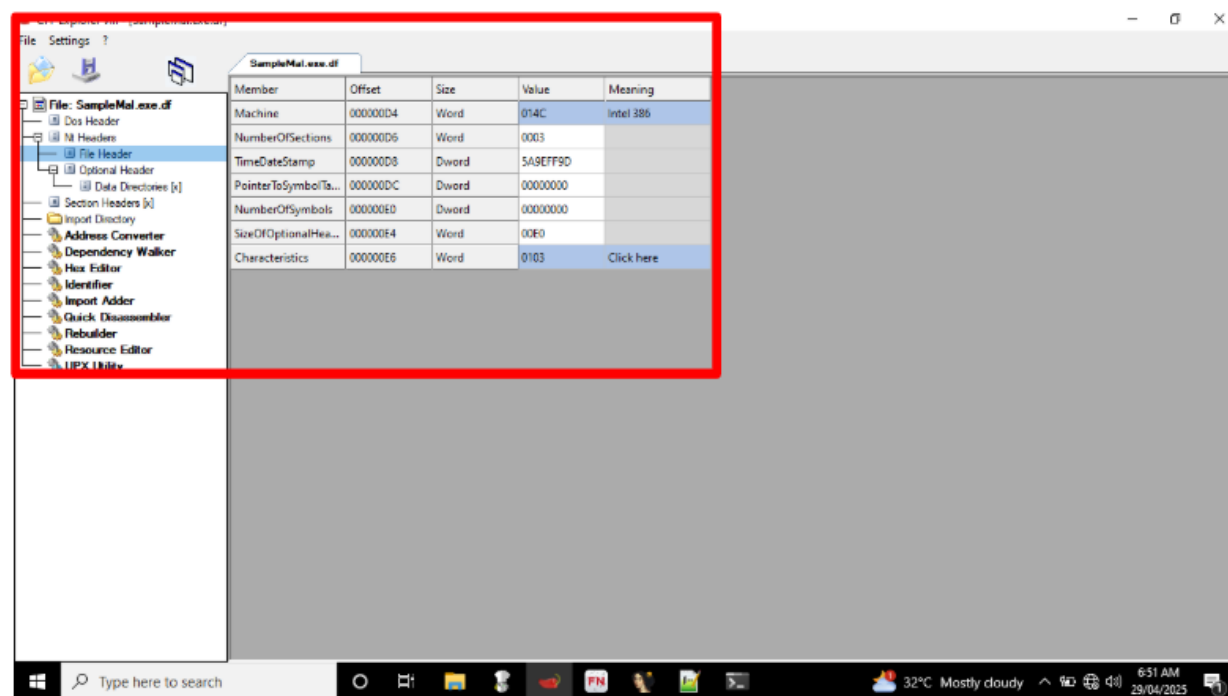


Figure 11: CFF Explorer Import Directory Overview

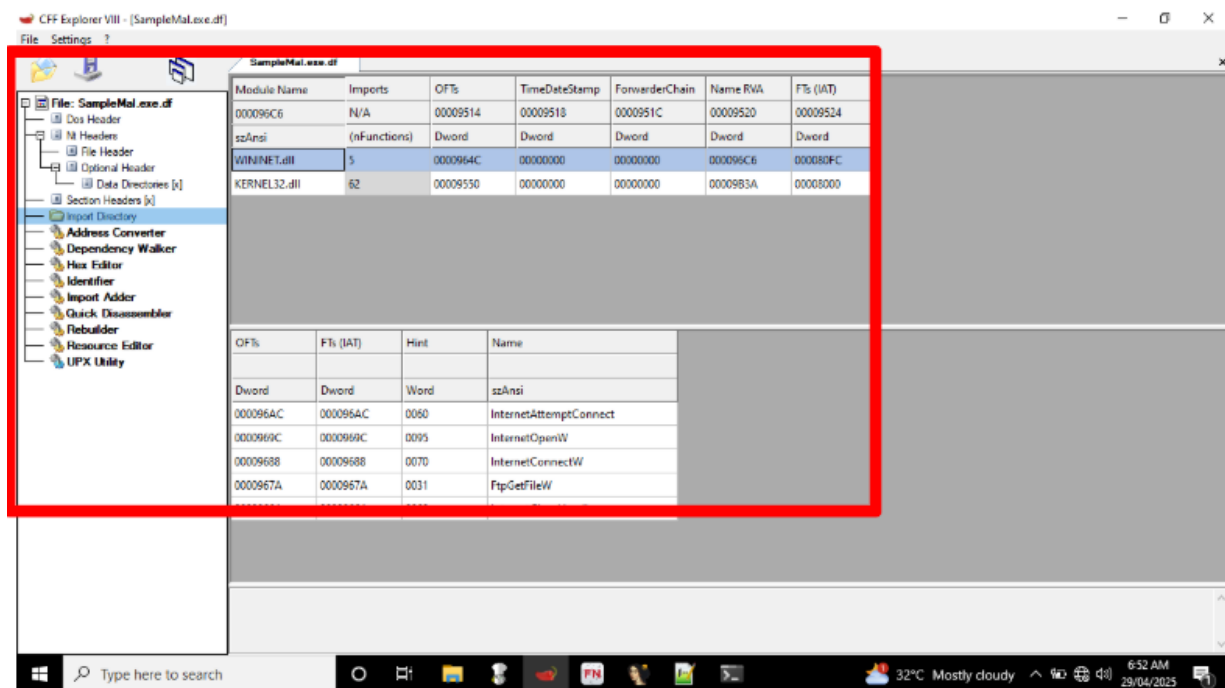


Figure 12: Table of API Functions and Malicious Usage

API Function	Potential Malicious Usage (per malAPI.io and MITRE ATT&CK)
InternetOpenW, InternetConnectW, InternetAttemptConnect, InternetCloseHandle, FtpGetFileW (WININET.dll)	T1071: Application Layer Protocol- Establish network connections for Command and Control (C2) or download additional payloads (Hassan, 2024).

UnhandledExceptionFilter, SetUnhandledExceptionFilter, RtlUnwind	T1562.001: Impair Defenses- Manipulate exception handling to disrupt analysis or hijack control flow.
IsDebuggerPresent	T1497.001: Virtualization/Sandbox Evasion- Detect debugger presence and alter behavior accordingly.
LoadLibraryA, GetProcAddress, GetModuleHandleA	T1055.001: Dynamic Linker Hijacking- Load and resolve additional modules and APIs dynamically to obfuscate malicious behavior.
WriteFile, GetStdHandle, GetFileType	T1005: Data from Local System- Potentially write exfiltrated or staged data to files or devices.
GetEnvironmentStringsA/W, FreeEnvironmentStringsA/W	T1083: File and Directory Discovery- Enumerate system environment variables for path manipulation or discovery.
InitializeCriticalSection, EnterCriticalSection, LeaveCriticalSection, DeleteCriticalSection, TlsAlloc, TlsGetValue, TlsSetValue, TlsFree, InterlockedIncrement, InterlockedDecrement	T1497: Virtualization/Sandbox Evasion- Thread-local storage and synchronization primitives can be used to obfuscate execution flow and complicate debugging.

InternetCloseHandle, FtpGetFileW (WININET.dll)	Command and Control (C2) or download additional payloads (Hassan, 2024).
CreateProcessW, DeleteFileW	T1105: Ingress Tool Transfer and T1561: Disk Wipe Launch new processes for payload execution and delete evidence (HP Development Company, L.P., 2025).
HeapAlloc, HeapFree, HeapCreate, HeapDestroy, HeapReAlloc, GetProcessHeap, HeapSize	T1027: Obfuscated Files or Information- Dynamic memory management for unpacking encrypted payloads at runtime.
VirtualAlloc, VirtualFree, VirtualProtect, VirtualQuery	T1055: Process Injection- Allocate and modify memory regions for code injection and execution.
GetTickCount, Sleep	T1497: Virtualization/Sandbox Evasion- Introduce execution delays to evade automated sandbox analysis (Miguel, 2025).
GetCurrentProcessId, GetCurrentProcess, TerminateProcess	T1489: Service Stop- Interact with and potentially terminate security-related processes.

GetLocaleInfoA, LCMapStringA/W, MultiByteToWideChar, WideCharToMultiByte, GetCPInfo, GetACP, GetOEMCP, GetStringTypeA/W	T1071: Application Layer Protocol (with possible localization targeting)- Adapt execution based on system locale, which can tailor attacks by region.
--	--

Figure 13: MITRE ATT&CK Matrix of SampleMal.Exe

Mitre Att&ck Matrix

Reconnai...	Resource Developm...	Initial Access	Execution	Persisten...	Privilege Escalation	Defense Evasion	Credential Access	Discovery	Lateral Movement	Collection	Command and Control	Exfiltration	Impact
Gather Victim Identity Information	Acquire Infrastructure	Valid Accounts	1 Native API	1 DLL Side-Loading	1 Process Injection	1 Process Injection	OS Credential Dumping	1 System Time Discovery	Remote Services	1 Archive Collected Data	1 Encrypted Channel	1 Exfiltration Over Alternative Protocol	Abuse Accessibility Features
Credentials	Domains	Default Accounts	Scheduled Task/Job	Boot or Logon Initialization Scripts	1 DLL Side-Loading	1 DLL Side-Loading	LSASS Memory	2 1 Security Software Discovery	Remote Desktop Protocol	Data from Removable Media	1 Non-Application Layer Protocol	Exfiltration Over Bluetooth	Network Denial of Service
Email Addresses	DNS Server	Domain Accounts	At	Logon Script (Windows)	Logon Script (Windows)	1 Obfuscated Files or Information	Security Account Manager	2 3 System Information Discovery	SMB/Windows Admin Shares	Data from Network Shared Drive	1 1 Application Layer Protocol	Automated Exfiltration	Data Encrypted for Impact

Figure 14: CC Explorer Section Headers Overview & Programmer Calculator

The screenshot displays the CC Explorer application window with the file `SampleMal.exe.dll` open. The left sidebar shows the file structure, with 'Section Headers [x]' selected. The main pane shows a table of section headers:

Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address	Linenumbers	Relocatio
00000218	00000220	00000224	00000228	0000022C	00000230	00000234	00000238
Byte[0]	Dword	Dword	Dword	Dword	Dword	Dword	Word
.text	000062C4	00001000	00007000	00001000	00000000	00000000	0000
.rdata	00001B48	00008000	00002000	00008000	00000000	00000000	0000
.idata	00003F84	0000A000	00002000	0000A000	00000000	00000000	0000

Below the table, a hex dump view shows the raw data for the selected section, with columns for Offset, Hex, and ASCII.

Overlaid on the right is the Windows Calculator in Programmer mode. The display shows the value `FFFF FFFF FFFF F2C4` in hexadecimal. The calculator interface includes various input bases (HEX, DEC, OCT, BIN), a numeric keypad, and logical/mathematical operation buttons.

Figure 15: Table of Sections in SampleMal.exe & Sizes

Section	Virtual Size (Hex)	Raw Size (Hex)	Virtual Size (Decimal)	Raw Size (Decimal)	Notes
.text	0x62C4	0x7000	25,508	28,672	Executable code section. Raw size > virtual size suggests padding (common). Padding can be used to hide injected shellcode or junk data.
.rdata	0x1B48	0x2000	6,472	8,192	Normal for a read-only data section. Raw size larger than the virtual size indicates standard alignment and padding. No major anomaly.
.data	0x2F84	0x2000	12,612	8,192	Suspicious: Virtual size > Raw size. Indicates the program allocates more writable memory at runtime than exists in the file, commonly used by malware for unpacking or runtime modification.

Figure 16: IDA Overview of `_main`

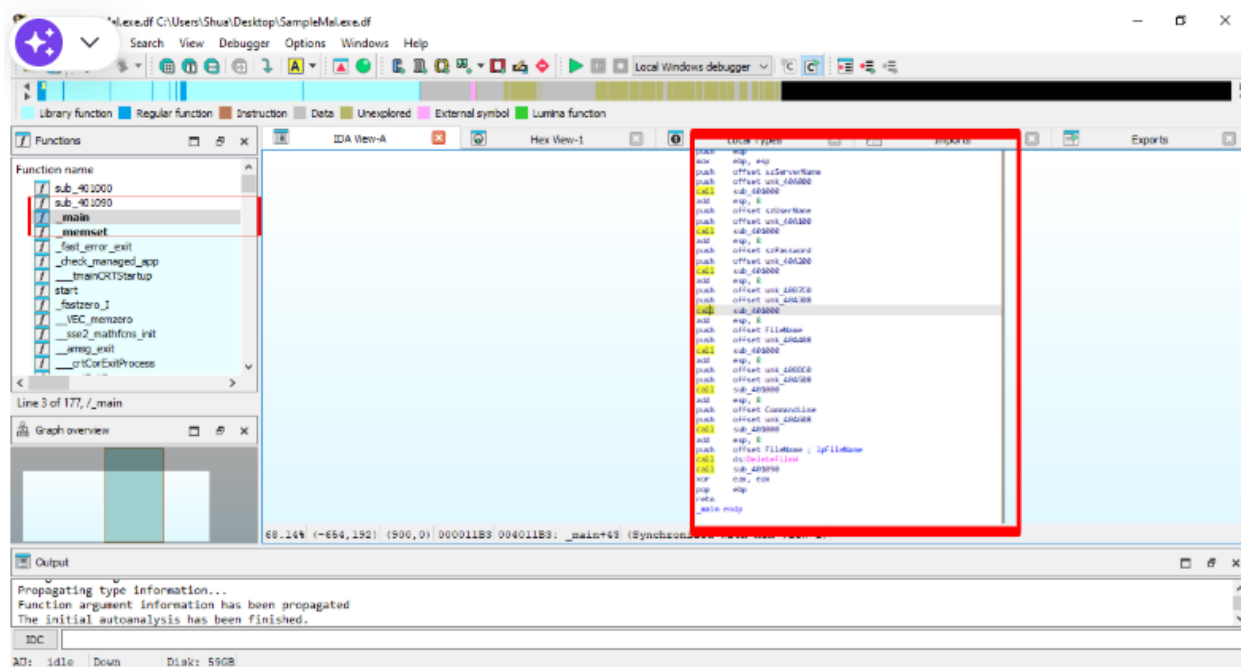


Figure 17: Ghidra Decompiler of FUN_00401000

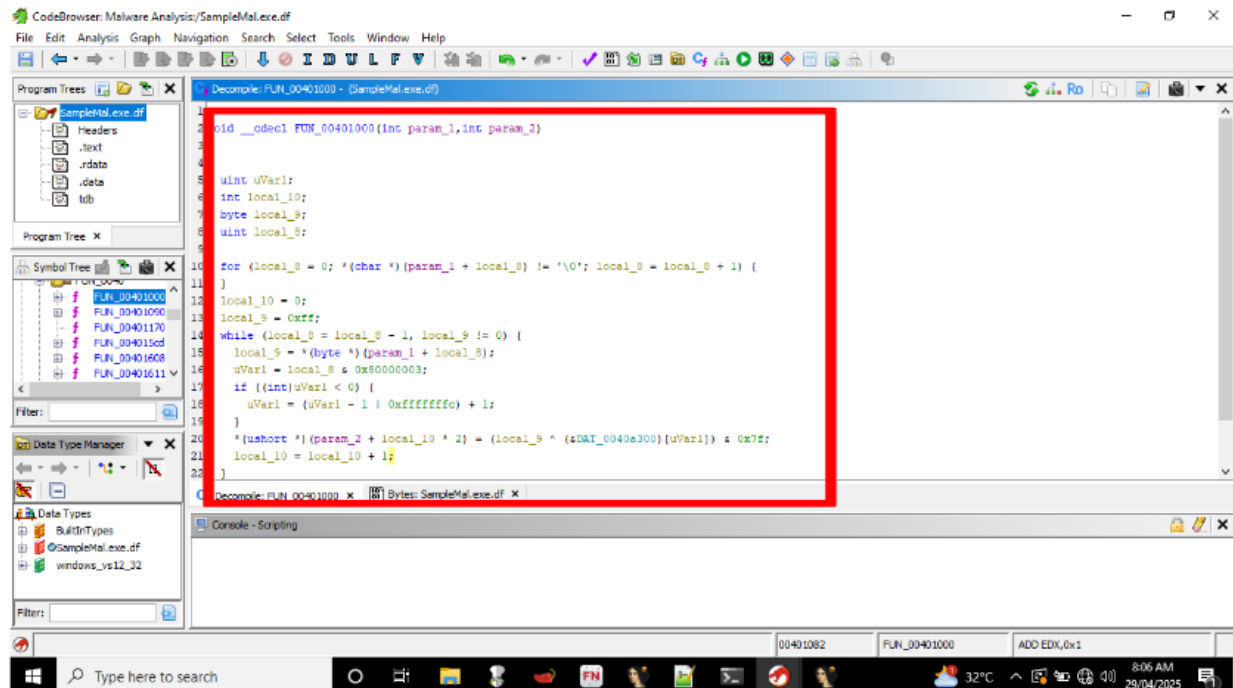


Figure 20: Table of Decrypted Content

Field	Decrypted Content
FTP Host	ftp.adrive.com
Username(s)	Petrov, pavel.gladyshec@ucd.ie
Password	Pa\$hka123
Searched File Names from \Petrov\Desktop*	minutesPim_9429.docx
	Payroll_Pamela5513.ppt
	receipt_Diana_377.pptx

Figure 21: Wireshark Overview of Filter: HTTP

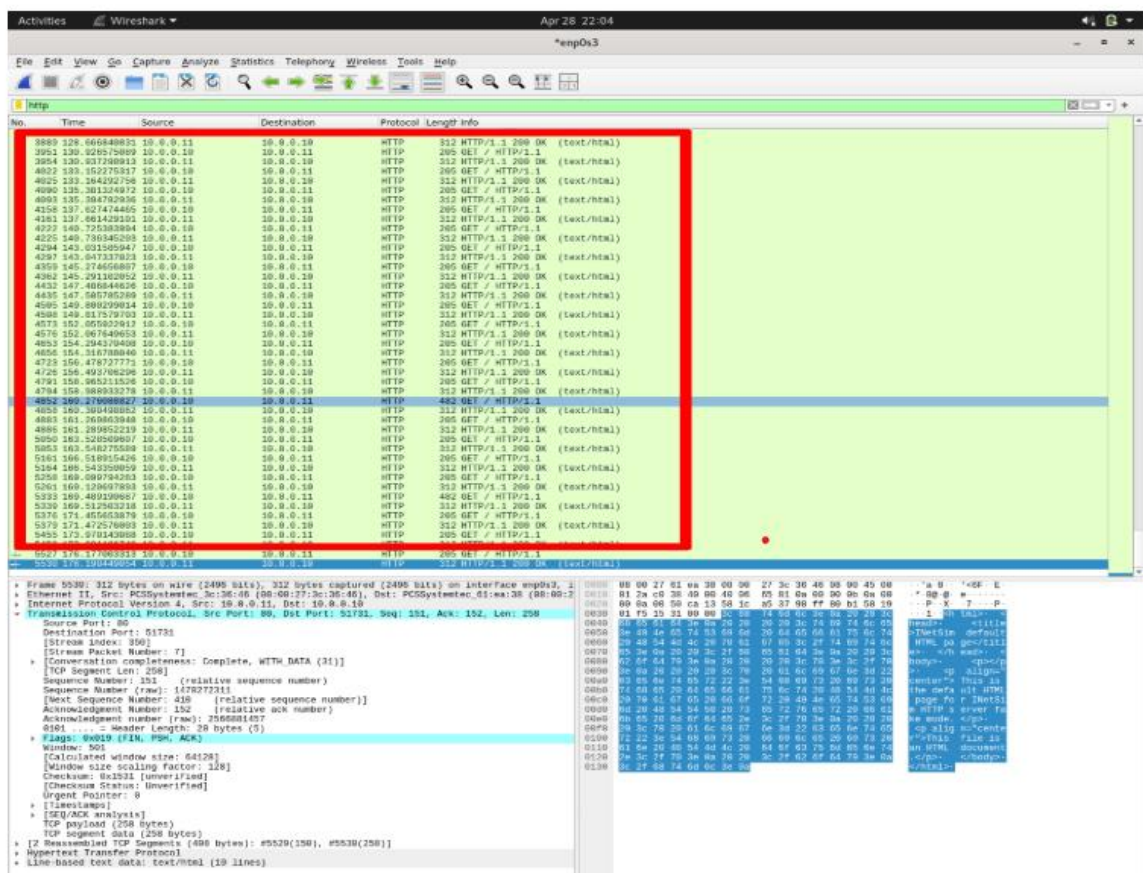


Figure 22: Wireshark Overview of Filter: FTP

Activities Wireshark Apr 28 21:53
Capturing from enp0s3

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

FTP

No.	Time	Source	Destination	Protocol	Length	Info
246	6.691524614	10.0.0.11	10.0.0.10	FTP	88	Response: 220 TNetSia FTP Service ready.
248	6.694798298	10.0.0.10	10.0.0.11	FTP	69	Request: USER pavel-gladyshev@ic
250	6.696409894	10.0.0.11	10.0.0.10	FTP	88	Response: 331 Please specify the password.
252	6.720217262	10.0.0.10	10.0.0.11	FTP	10	Request: 252 PORT 255,2010x12
254	6.699060892	10.0.0.11	10.0.0.10	FTP	77	Response: 230 Login successful.
256	6.710456598	10.0.0.10	10.0.0.11	FTP	62	Request: TYPE I
258	6.712874644	10.0.0.11	10.0.0.10	FTP	85	Response: 200 Switching to BINARY mode.
260	6.714015723	10.0.0.10	10.0.0.11	FTP	60	Request: PASV
262	6.715105284	10.0.0.11	10.0.0.10	FTP	76	Response: 500 Unknown command.
264	6.720323729	10.0.0.10	10.0.0.11	FTP	62	Request: TYPE I
265	6.721498151	10.0.0.11	10.0.0.10	FTP	85	Response: 200 Switching to BINARY mode.
267	6.723468678	10.0.0.10	10.0.0.11	FTP	76	Request: PORT 18,0,9,10,106,158
268	6.726269416	10.0.0.11	10.0.0.10	FTP	84	Response: 200 PORT command successful.
271	6.728894748	10.0.0.10	10.0.0.11	FTP	70	Request: SIZE 28944.exe
272	6.729834981	10.0.0.11	10.0.0.10	FTP	76	Response: 500 Unknown command.
274	6.731506818	10.0.0.10	10.0.0.11	FTP	70	Request: RETR 28944.exe
275	6.733197644	10.0.0.11	10.0.0.10	FTP	80	Response: 550 Failed to open file.

Frame 252: 78 bytes on wire (608 bits), 78 bytes captured (608 bits) on interface enp0s3, id 0
 Ethernet II, Src: PCSSystemtec 61:aa:38 (08:00:27:61:aa:38), Dst: PCSSystemtec 3c:38:46 (08:00:27:3c:38:46)
 Internet Protocol Version 4, Src: 10.0.0.10, Dst: 10.0.0.11
 Transmission Control Protocol, Src Port: 58332, Dst Port: 21, Seq: 30, Ack: 67, Len: 18
 File Transfer Protocol (FTP)
 [Current working directory:]

0000 08 00 27 3c 38 46 08 00 27 61 aa 38 08 00 45 00 ...c6...a 8...E
 0010 08 38 07 11 40 00 08 06 0f 8a 8a 00 00 8a 0a 00 ...B 8...
 0020 09 00 c4 9c 00 15 64 8f c5 5c 0b 28 ad 8e 59 19S...A(-P
 0030 83 ff c1 6f 00 00 58 41 53 53 29 50 61 24 68 0bPA SS PaShk
 0040 61 31 32 33 0d 0a a123...

Figure 23: Analysis & Interpretation Table of Host Behaviors

Behavior	Registry/File/System Component	Suspicious Indicator	Analysis and Interpretation
Registry Query	HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Internet Settings\TcpAutotuning		Indicates potential attempts to inspect or manipulate TCP autotuning settings. This may be used to alter network behavior for stealthy communications or to optimize command-and-control (C2) traffic.
Registry Query	DNS-related keys (e.g., DNS timeout parameters)		Suggests the malware may be modifying DNS resolution behavior to evade detection, accelerate callback attempts, or support Fast Flux techniques.
File Creation	Files written or modified within C:\Windows\SysWOW64\ (e.g., DLLs such as cryptbase.dll)		Writing to system directories, especially SysWOW64, is highly suspicious. This behavior may indicate DLL side-loading, hijacking, or persistent code execution.
File Mapping	CreateFileMapping and MapViewOfFile involving DLLs		Memory-mapped files involving DLLs can be used for stealthy in-memory execution, code injection, or fileless malware operations.
Registry Access	Repeated use of RegOpenKey and RegCloseKey on TCP/IP parameters or autorun keys		Indicates reconnaissance activity. May precede registry modification to establish persistence, disable security features, or modify startup behavior.
File Access	Creation or modification of files in temporary or unexpected locations		May indicate payload staging or unpacking of components. Requires correlation with process behavior and file content.

Figure 24: Cmder Overview of Nano

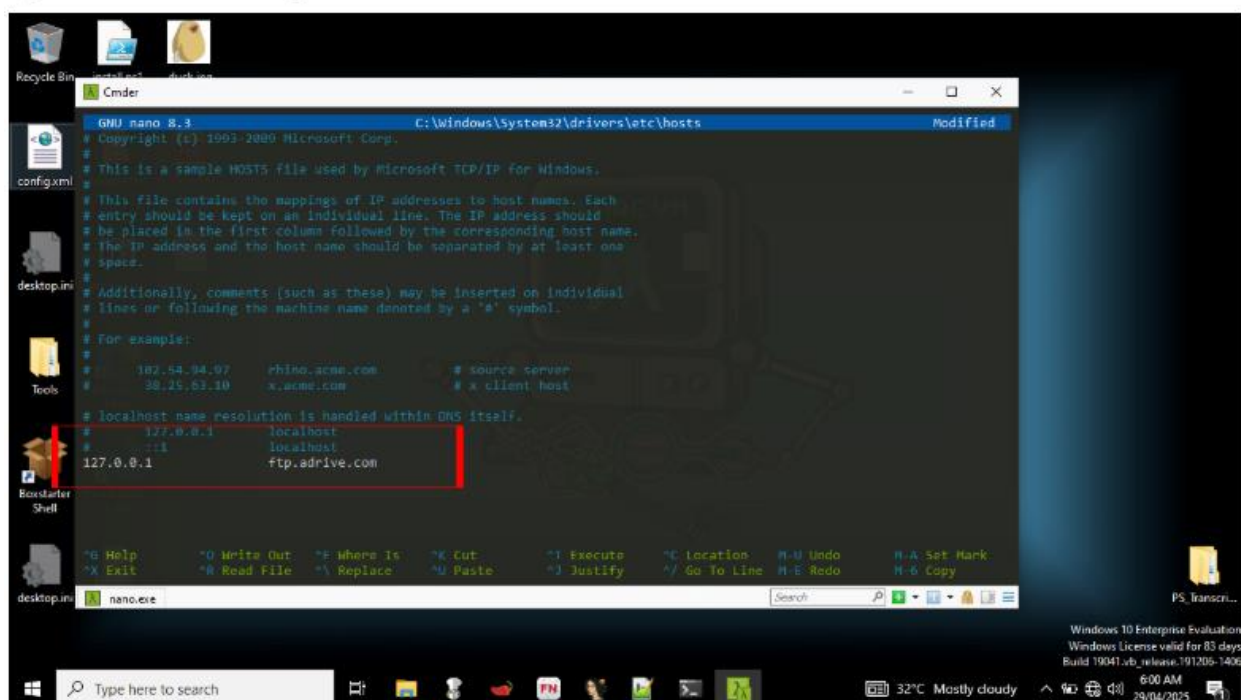


Figure 25: ProcMon Overview of Initial Detonation

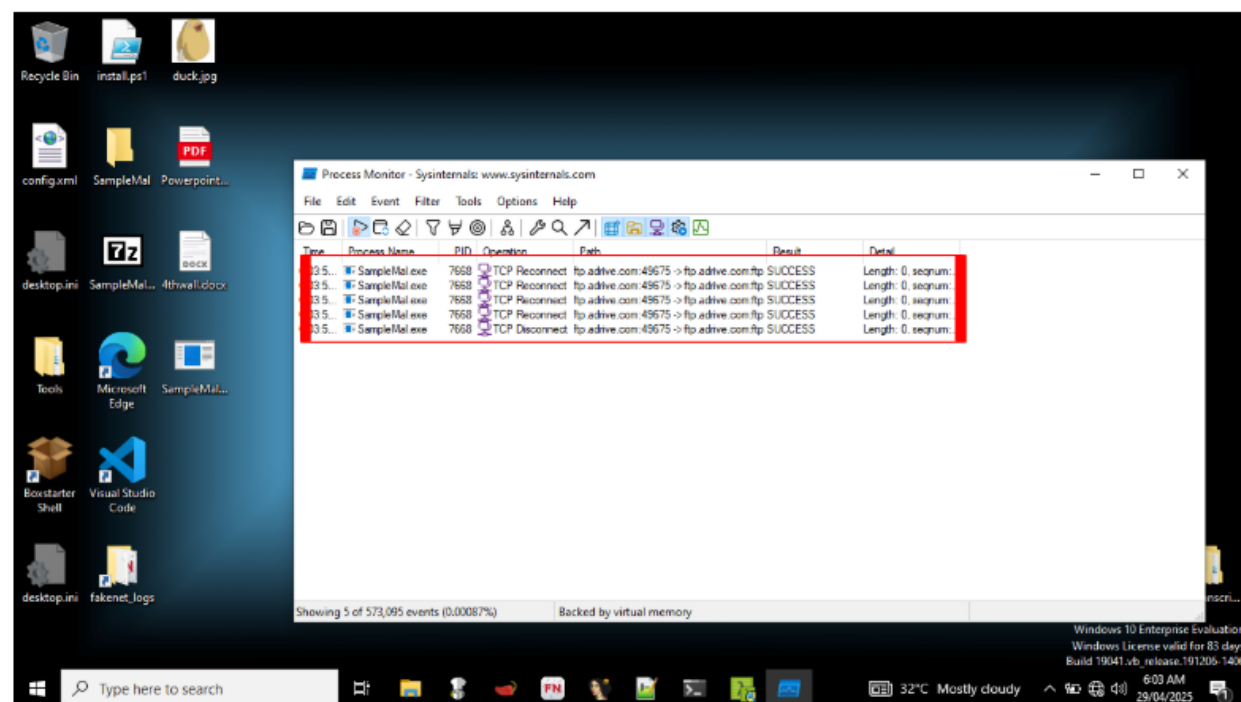


Figure 26: ProcMon & Cmdr Overview of NetCat

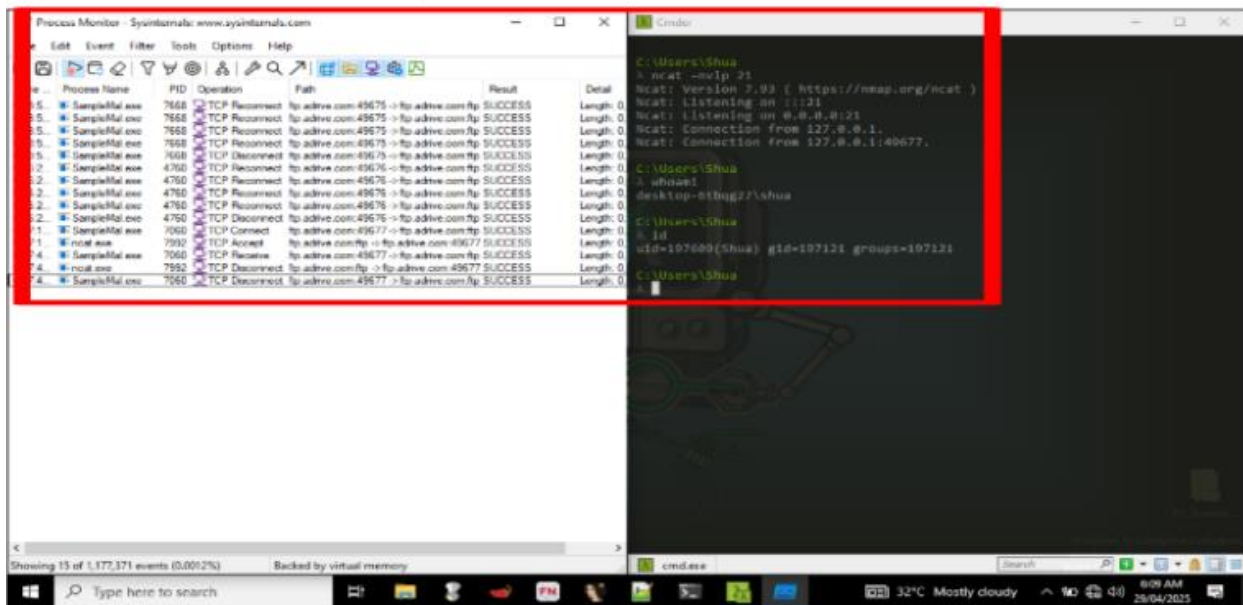


Figure 27: Classification Graph of SampleMal.exe

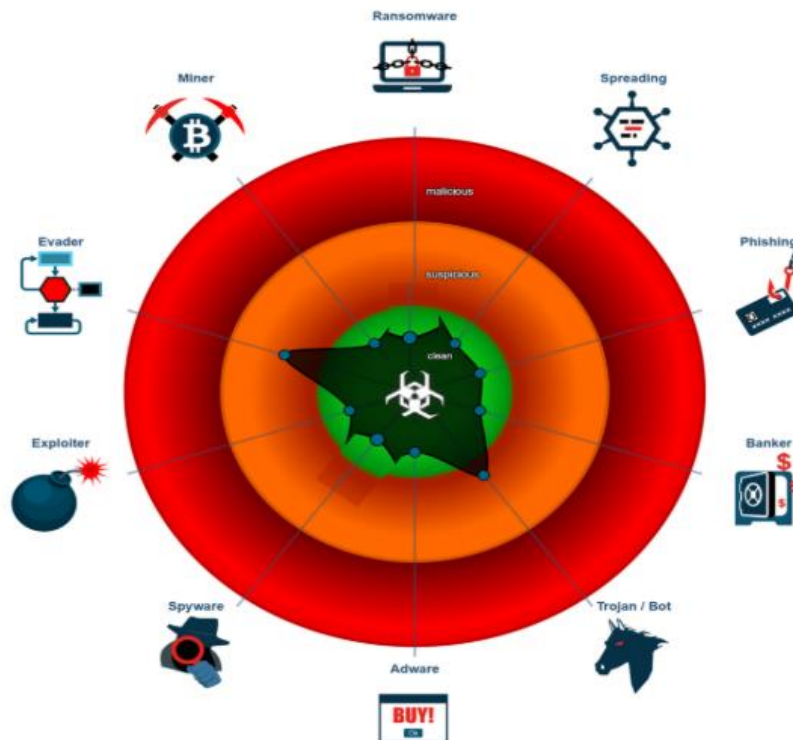


Figure 28: IDA Overview of `_main`

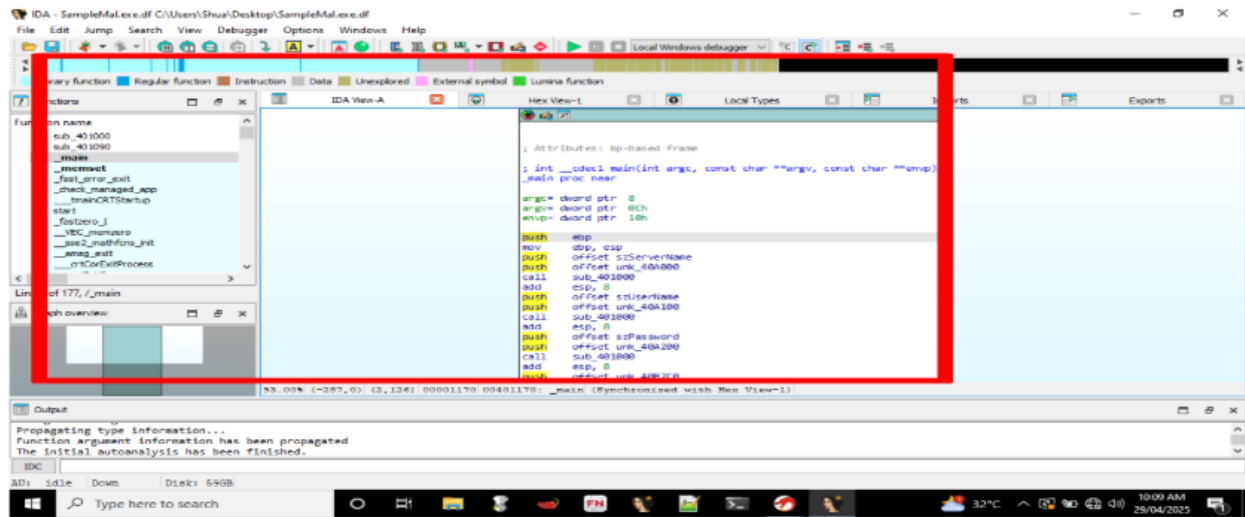


Figure 29: IDA Overview of Function `sub_401000`

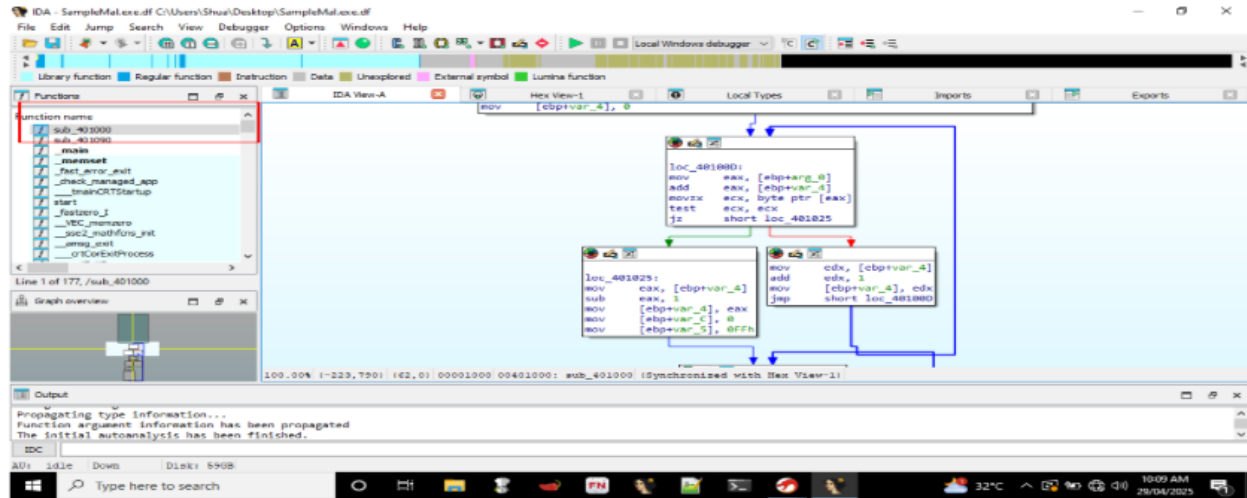


Figure 30: Control-Flow Graph of Function `sub_401090`

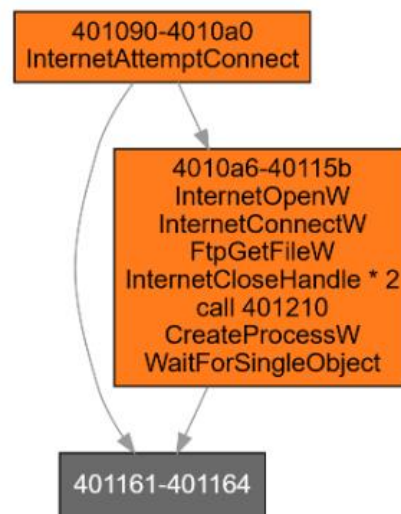


Figure 31: Control-Flow Graph of Function sub_401170

